



UNIVERSIDAD DE ALCALÁ

ESCUELA POLITÉCNICA SUPERIOR

Máster Universitario en Ciencia y Tecnología desde el Espacio

Trabajo Fin de Máster

Creación de un modelo en TASTE para el soporte de servicios PUS

Author: Jorge Urbelz Alonso-Cortés

Tutor: Óscar Rodríguez Polo

20 de septiembre de 2026

UNIVERSIDAD DE ALCALÁ

ESCUELA POLITÉCNICA SUPERIOR

Máster Universitario en Ciencia y Tecnología desde el Espacio

Trabajo Fin de Máster

Creación de un modelo en TASTE para el soporte de servicios PUS

Autor: Jorge Urbelz Alonso-Cortés

Tutor: Óscar Rodríguez Polo

Tribunal:

Presidente: Pablo Parra Espada

Vocal 1º: Agustín Martínez Hellín

Vocal 2º: Óscar Rodríguez Polo

Fecha de depósito: 20 de septiembre de 2026

"Tengo mi licencia, doctor, pero en mis últimos textos no hay nada referente a hipermotores ni curvonavegación. Sólo el corriente juego de niños de las tres dimensiones"

Yo, Robot, Isaac Asimov

Agradecimientos

Este trabajo es el fruto de muchas horas de trabajo. Ha sido un camino largo, en el cual aparecieron ingentes problemas a su paso. En el momento de mirar atrás y ver todo lo que he tenido que afrontar, me quedo con la satisfacción de haber hecho todo lo que estaba en mi mano. Quiero agradecer a mi familia por las horas que han tenido que soportar de reproches y quejas por mi parte. Un trabajo no es bueno si uno no siente orgullo de lo realizado. Yo me voy muy orgulloso.

Mi contacto Jorge Urbelz Alonso-Cortés <jorge.urbelz@edu.uah.es>, juacmola@gmail.com.

Resumen

Los sistemas embebidos en tiempo real surgieron en la segunda mitad del siglo XX para controlar procesos físicos complejos de forma autónoma, precisa y con una respuesta garantizada dentro de límites de tiempo estrictos. Se pueden encontrar desde electrodomésticos hasta vehículos, generalmente en dispositivos cuyo aspecto no suele estar asociado a una computadora.

En una nave espacial, existen una serie de sistemas embebidos encargados de determinar y controlar la navegación, orientación, comunicación y el envío de telemandos y telemetría. La ciencia vinculada al desarrollo y aplicación de nuevas tecnologías en los sistemas electrónicos es conocida como astriónica o electrónica espacial.

The ASSERT Set of Tools for Engineering (TASTE) es un programa creado por la European Space Agency (ESA) junto a una serie de socios de la industria espacial, que se dedica al diseño, evaluación, corrección de software, generación de documentación y automatización de sistemas embebidos en tiempo real. La herramienta al completo puede ser utilizada por todo el mundo, evidenciando el papel fundamental del desarrollo de nuevas ideas en el ámbito espacial. Permite poder crear desde cero diseños de código de bajo nivel para microcontroladores y sistemas de comunicación distribuidos. En el espacio, el número de plataformas hardware utilizadas es amplio, por lo que es capaz de soportar: x86/Linux, Raspberry Pi y LEON2/LEON3 con RTEMS, entre otros.

En este Trabajo Fin de Máster se construirá un modelo encargado de digerir telemandos y de generar telemetrías. A lo largo de este documento se darán detalles acerca de los cimientos sobre los que se basa TASTE, se explicará el desarrollo del proyecto, se mostrarán los resultados y se mostrarán las conclusiones finales.

Palabras clave: TASTE, servicios PUS, telemando, telemetría, EDROOM.

Abstract

Real-time embedded systems emerged in the second half of the 20th century to control complex physical processes autonomously, accurately, and with a guaranteed response within strict time limits. They can be found in everything from household appliances to vehicles, generally in devices whose appearance is not usually associated with a computer.

A spacecraft contains several embedded systems responsible for determining and controlling navigation, orientation, communication, and the transmission of telecommands and telemetry. The science relating to the development and application of new technologies in electronic systems is known as astrionics, or space electronics.

The ASSERT Set of Tools for Engineering (TASTE) is a program created by the European Space Agency (ESA) in collaboration with a number of partners from the space industry, dedicated to the design, evaluation, debugging, documentation, and automation of real-time embedded systems. The complete tool is available for use by anyone, highlighting the vital role of developing new ideas in the space sector. It enables the creation of low-level code designs from scratch for microcontrollers and distributed communication systems. In the space sector, a wide range of hardware platforms are used, so the tool supports, amongst others: x86/Linux, Raspberry Pi and LEON2/LEON3 with RTEMS.

In this Master's Thesis, a model will be developed to process remote control signals and generate telemetry data. Throughout this document, details will be provided regarding the foundations on which TASTE is based; the development of the project will be explained; the results will be presented, and conclusions will be drawn.

Keywords: TASTE, PUS services, telecommand package, telemetry package, EDROOM.

Índice general

Agradecimientos **vii**

Resumen **ix**

Abstract **xi**

Índice general **xiii**

Índice de figuras **xv**

Índice de tablas **xvii**

Índice de listados de código fuente **xix**

Lista de acrónimos **xxii**

Lista de símbolos **xxii**

1 Introducción **1**

2 Estado del arte **3**

2.1 Sistema de comunicación espacial 3

2.2 Servicios PUS 4

2.2.1 Telemando 8

2.2.2 Telemetría 9

2.3 TASTE 10

2.3.1 SDL 11

2.3.2 ASN.1 11

3 Análisis de objetivos y metodología **15**

3.1 Objetivos 15

3.2 Metodología de trabajo 16

3.3 Requisitos del sistema 17

3.4 Herramientas y recursos 17

4 Desarrollo **19**

4.1 Arquitectura del proyecto 19

4.2 Inyección de los telemandos 26

4.3 Administrador de telemandos 28

4.4 HKFDIRMng 31

4.5 BKGTCExec 33

4.6 Variables ASN.1 del proyecto 33

5 Resultados **35**

5.1 Validación de los servicios PUS 01 y 17 35

5.1.1 Resultado esperado 37

5.1.2 Resultados de la validación 38

5.2 Validación de los servicios PUS 03 41

5.2.1 Resultado esperado 41

5.2.2 Resultados de la validación 42

5.3 Validación de los servicios PUS 20 44

5.3.1 Resultado esperado 45

5.3.2 Resultados de la validación 45

5.4 Validación de FDIR 47

5.4.1 Resultado esperado 47

5.4.2 Resultados de la validación 48

5.5 Conclusiones 50

6 Conclusiones y líneas futuras **51**

Bibliografía **53**

Índice de figuras

2.1 Intercambio de mensajes entre el control desde tierra y la nave espacial. Desde el control de tierra se mandan los comandos empaquetados en telemandos. La nave espacial recopila datos de sensores e instrumentos y los transmite a través de la telemetría 4
2.2 El *Space Packet Protocol* de CCSDS definida en *CCSDS 133.0-B-2*[7] 5
2.3 Estructura interna del campo *packet secondary header* de un telemando[8] 8
2.4 Estructura interna del campo *user data* de un telemando[8] 9
2.5 Estructura interna del campo *packet secondary header* de una telemetría[8] 10
2.6 Estructura interna del campo *user data* de un telemetría[8] 10
2.7 Diagrama básico de transición SDL[14] 11

3.1 Comparación de la arquitectura antigua y nueva 16

4.1 Arquitectura principal del proyecto 20
4.2 Ventana de configuración de un componente o *function* 21
4.3 Ventana de configuración de una interfaz 22
4.4 Diagrama de secuencia general de la interfaz *poll* 23
4.5 Diagrama de secuencia en caso de que el telemando sea prioritario 23
4.6 Diagrama de en caso de que el telemando provenga de *housekeeping* o FDIR 24
4.7 Diagrama de secuencia en caso de que el telemando se vaya a ejecutar en segundo plano . 25
4.8 Diagrama de secuencia en caso de que el telemando de que se haya que realizar un reinicio 25
4.9 Diagrama de secuencia general de la interfaz *trigger* 26
4.10 Diagrama de estados del componente TCManager 28
4.11 Proceso interno del componente TCManager en System Design Languages (SDL) 29
4.12 Diagrama de estados del componente HKFDIRMng 32
4.13 Proceso interno del componente HKFDIRMng en SDL 32
4.14 Proceso interno del componente BKGTCExec 33

5.1 Inicialización del terminal al comenzar la ejecución 38
5.2 Se inyectan los telemandos TC[17,1] y TC[17,3] 39
5.3 Se inyectan varios telemandos TC[17,1] 40
5.4 Telemetrías en serie que aparecen tras la inyección de telemandos de la imagen 5.3 40
5.5 Se inyecta el telemando TC[3,6] 43
5.6 Se inyecta el telemando TC[3,5] 43
5.7 Se inyecta el telemando TC[3,31] 44
5.8 Se inyectan los telemandos TC[20,3] y TC[20,1] 46
5.9 Inyección de los telemandos TC[20,3], TC[12,5], TC[19,1], TC[5,5], TC[19,4] y TC[12,1] . 49
5.10 Salida esperada 49

Índice de tablas

2.1 Descripción de cada uno de los servicios PUS[8] 5
2.2 Algunos tipos de datos en ASN.1 12

4.1 Algunos posibles parámetros del *System Data Pool* 32

5.1 Telemandos y telemetrías esperados 37
5.2 Telemandos y telemetrías esperados 42
5.3 Telemandos y telemetrías esperados 45
5.4 Telemandos y telemetrías esperados 48

Índice de listados de código fuente

2.1 Ejemplo de creación de alguna variables en código ASN1 13
4.1 La función *EmuGSS_SendProgrammedTCs()* recupera los telemandos inyectados 27
4.2 Inicialización de TCManager 28
4.3 Leer la cola de mensajes de TCManager 30
4.4 Gestión del tipo de comando por parte de TCManager y Legacy 31
4.5 Variables generadas en ASN.1 33
5.1 Validación de los servicios PUS 01 y 17 36
5.2 Validación de los servicios PUS 03 41
5.3 Validación de los servicios PUS 20 44
5.4 Validación de FDIR 46

Lista de acrónimos

AADL	Architecture Analysis & Design Language.
ASN.1	Abstract Syntax Notation One.
CCSDS	Consultative Committee for Space Data Systems.
CDS	Segmento de Día CCSDS.
CNSA	Administración Espacial Nacional de China.
CPU	Unidad Central de Procesamiento.
ECSS	European Cooperation for Space Standardization.
ESA	European Space Agency.
FDIR	Detección, Aislamiento y Recuperación de Fallos.
GUI	Interfaz Gráfica de Usuario.
ICU	Unidad de Control de un Instrumento.
IRQ	Solicitud de Interrupción.
ISO	Organización Internacional de Normalización.
ITU	Unión Internacional de Telecomunicaciones.
JAXA	Agencia Japonesa de Exploración Aeroespacial.
JSON	Notación de Objeto de JavaScript.
MAST	Modeling and Analysis Suite for Real-Time Applications.
MBSE	Ingeniería de Software Basada en Modelos.
NASA	Administración Nacional de Aeronáutica y el Espacio.
OBT	On-Board Time.
PID	Parameter IDentifier.
PMONID	Parameter Monitoring ID.
PUS	Packet Utilization Standard.
RAE	Real Academia Española.
RF	Radio Frecuencia.

SCTRSE	Software de Control en Tiempo Real para Sistemas Espaciales.
SDL	System Design Languages.
SID	Structure ID.
SO	Sistema Operativo.
SRG	Space Research Group.
TASTE	The ASSERT Set of Tools for Engineering.
TT&C	Telemetry, Tracking and Command.
UAH	Universidad de Alcalá.
XML	Lenguaje de Marcas Extensible.

Capítulo 1

Introducción

Los servicios Packet Utilization Standard (PUS) de la ESA nacieron con el objetivo de definir un conjunto de servicios estandarizados para el control, monitorización y gestión de satélites y naves espaciales a nivel de aplicación. Su uso se valora en la transacción de telemandos y telemetrías entre los puestos de escucha en la Tierra y los dispositivos que se encuentran en el espacio.

El lenguaje entre los humanos requiere el uso de ciertos estándares de comunicación a la hora de intercambiar información entre unos y otros. Al abrir una conversación, debemos saludar y esperar el cumplimiento de la otra persona o grupo de personas. Una vez realizadas las presentaciones, comienza el intercambio de datos entre unos y otros. En el espacio pasa lo mismo: cuando se envía una orden a cualquier objeto, se debe encapsular dicho comando dentro de un telemando, mientras se espera la recepción de telemetría que contenga la respuesta a dicha orden o cierta información de interés.

A pesar de que la simbiosis que se produce entre los telemandos y telemetrías puede parecer suficiente, se quiso ahondar en la manera en que se dividen estos mensajes. De esta manera, aquellos que se encarguen de configurar un parámetro de una señal en concreto, llevarán un código en el telemando distinto al mensaje que pide un informe acerca de la memoria de la nave. Por otro lado, cada mensaje tiene una estructura de datos a enviar diferente. En definitiva, los servicios PUS suponen las reglas más básicas para la creación de un lenguaje propio entre los dos extremos de una misión espacial: el operador de Tierra y el dispositivo final. El correcto entendimiento entre ambos sujetos será esencial para el control de la misión, pues se evitan pérdidas de tiempo innecesarias y problemas de comunicación.

La Ingeniería de Software Basada en Modelos (MBSE) es un enfoque de desarrollo de ingeniería de sistemas que busca construir y explotar modelos de dominio. Este enfoque se apoya en varias herramientas de ingeniería, como el conjunto de herramientas TASTE, respaldado por la ESA.

TASTE se especializa en diseño, validación y verificación de software, así como en la generación de aplicaciones binarias para sistemas nativos y embebidos. El diseño de software es heterogéneo y utiliza una amplia gama de lenguajes de modelado y programación, estructurados en diversas vistas, como arquitectura funcional, comportamiento de componentes e implementación. La validación del software incluye análisis de tipos estáticos, análisis de planificación en tiempo real, simulación guiada/aleatoria, depuración y pruebas.

Uno de los fuertes a destacar de TASTE es que permite y proporciona automatización para la mayoría de las fases del desarrollo de sistemas de software: categoriza sistemas heterogéneos mediante varios lenguajes de modelado y programación, tiene una correcta generación de código, compilación e implementación de aplicaciones binarias, y permite la validación mediante análisis estático y simulación. [1]

En el siguiente capítulo se documentará aquellas investigaciones sobre las que parte el proyecto. Además, se profundizará en el manejo de servicios PUS, la construcción de telemandos y telemetrías, así como el funcionamiento de TASTE y sus herramientas. En el tercer capítulo, veremos los distintos objetivos llevados a cabo durante la realización de este trabajo, así como la metodología presente. Después, pasaremos a desarrollar la arquitectura idealizada. Terminaremos mostrando los resultados experimentados del programa y las conclusiones a las que se llega y las vías futuras de progreso a partir del proyecto creado.

Capítulo 2

Estado del arte

A la hora de iniciar un proyecto de esta índole, debemos fijarnos en el marco teórico construido en este campo. En este caso, no empezamos desde cero, sino que partimos de una base muy teorizada por otros investigadores y expertos. Partiendo de la idea de nuestro proyecto, se debe entender primero qué es un telemando y qué es una telemetría, cómo es la estructura de la que se componen y las distintas maneras en que las máquinas tienen para interpretarlos y administrarlos. Por otro lado, se explicará la herramienta TASTE y se ahondará en cada uno de los detalles que esconde: ASN.1y SDL. Se expondrá el contexto suficiente en este capítulo para que el lector pueda partir con una idea clara de lo que está por venir en el resto de capítulos.

2.1 Sistema de comunicación espacial

La comunicación espacial se rige por el sistema Telemetry, Tracking and Command (TT&C), el cual debe asegurar que el sistema de vuelo de la nave u objetivo depositado en el espacio opera correctamente. La comunicación con las estaciones de tierra emplea ondas de Radio Frecuencia (RF) que se envían y reciben a través del sistema de antenas. Como bien indica el nombre, la funcionalidad se divide en tres aspectos definitorios: *Telemetry* (monitorea los sensores e instrumentos de la nave y envía informes a los ingenieros), *Tracking* (aporta información sobre la dinámica de vuelo de la nave) y *Command* (los comandos enviados desde tierra). En este trabajo nos vamos a centrar en la telemetría y en los comandos (se empleará el nombre de telemando a partir de ahora).[2]

A pesar de que en este trabajo se hará mención permanente al sistema de comunicación espacial entre una nave y las estaciones terrestres, cabe mencionar que este sistema no es único de este escenario, pues se puede extrapolar a distintas situaciones y con otros actores. La nave puede ser una nave espacial, un satélite de cualquier tipo (geostacionario, de órbita baja, etc) o una estación espacial. En el futuro, cuando se construyan estaciones permanentes en la Luna, estas usarán este sistema, aunque sea necesario hacer uso de satélites como repetidores de la señal.

Con el objetivo de cumplir con la adopción de este sistema en todo el mundo, las grandes agencias espaciales como la ESA, Administración Nacional de Aeronáutica y el Espacio (NASA), Agencia Japonesa de Exploración Aeroespacial (JAXA) y Administración Espacial Nacional de China (CNSA) acordaron su estandarización en los procesos de ingeniería. La ESA, a través de la iniciativa European Cooperation for Space Standardization (ECSS), ha oficializado el sistema mediante la creación de nuevos estándares. En concreto, emplea:

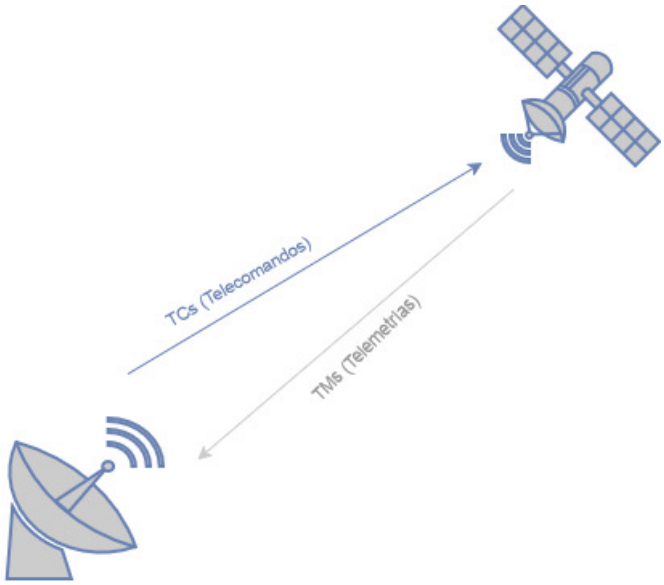


Figura 2.1: Intercambio de mensajes entre el control desde tierra y la nave espacial. Desde el control de tierra se mandan los comandos empaquetados en telemandos. La nave espacial recopila datos de sensores e instrumentos y los transmite a través de la telemetría

- *ECSS-E-ST-50C - Rev.2* para especificar los requisitos de desarrollo de las comunicaciones de datos extremo a extremo (la terminología y actividades a utilizar en la ingeniería de sistemas y los requisitos de los sistemas de comunicación espacial con respecto a la funcionalidad y el rendimiento).[3]
- *ECSS-E-AS-50-24C Rev.1* y *ECSS-E-AS-50-21C Rev.2* explican las modulaciones, formatos de código de los canales y las asignaciones de frecuencia de telemandos y telemetrías.[4][5]

Como se ha explicado, este sistema es capaz de adaptarse a situaciones de comunicación de extremo a extremo sin demasiada dificultad, como es el caso del crecimiento de las constelaciones satelitales, las cuales está provocando pérdidas en la comunicación tradicional debido, principalmente, al bajo arco visible del satélite y al retardo que se produce en la señal. Una solución a este problema puede conseguirse si, en vez de considerar a cada satélite de la constelación, entendemos las constelaciones satelitales como una red única. Aprovechando la sinergia que se genera, se puede conseguir mejorar el sistema.[6]

2.2 Servicios PUS

Los servicios PUS son una serie de estándares, apoyados por la Consultative Committee for Space Data Systems (CCSDS)[7] y la ECSS[8], de protocolos a nivel de aplicación entre control de tierra y el espacio, en orden de satisfacer la integración eléctrica, las pruebas y las operaciones de vuelo. Se establecen un total de veintitrés servicios que engloban todas las operaciones comunes de control de satélites. El término *Utilisation* se integra en el título debido a que la intención al usar estos servicios es poder abordar todos los aspectos que están relacionados con el uso de los paquetes: las circunstancias en las que se generan, sus reglas para realizar el intercambio, la estructura, el formato y su contenido. [9]

En la tabla 2.1 se muestra el código y una descripción de los servicios. Los servicios del estándar *ECSS-E-ST-70-41C* engloban una serie de telemandos y telemetrías existentes. El espacio que ocupan los servicios reservados está pensado para ser rellenado por los futuros servicios que están por crearse. También existe la posibilidad de crear servicios específicos para una misión. El estándar indica que aquellos servicios deberán tener un identificador superior a *127*, llegando a un máximo de *255*.

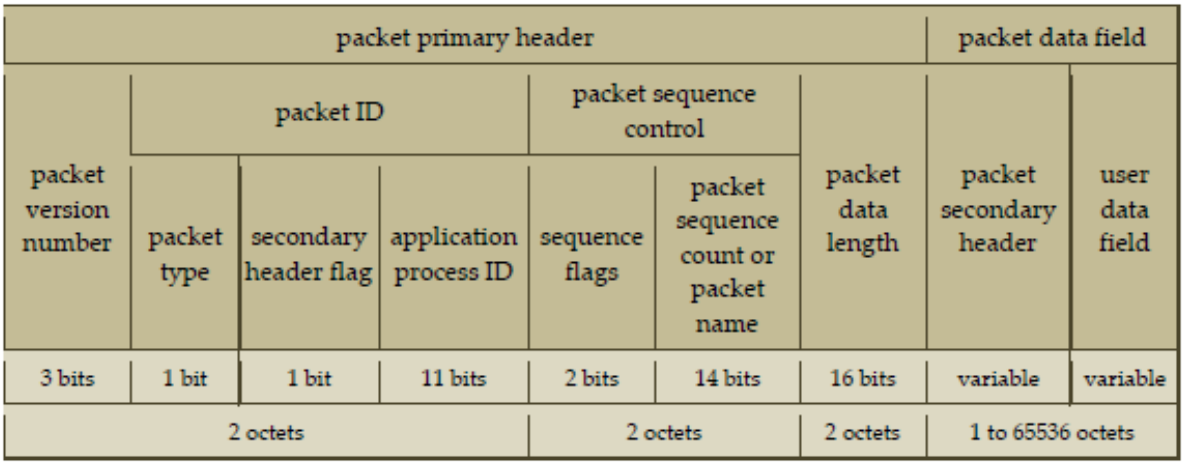


Figura 2.2: El *Space Packet Protocol* de CCSDS definida en *CCSDS 133.0-B-2*[7]

Para saber el tipo concreto de telemando o telemetría que se está enviando, los servicios cuentan con un identificador del tipo y otro para el subtipo. Por ejemplo, el tipo de servicio 02 sirve para acceder a un dispositivo. Dentro de este tipo de servicio, existe un total de once subtipos. En concreto, ocho de ellos están pensados para ser enviados en un telemando, mientras que el resto están ideados para telemetrías. El telemando TC[2,8] está constituido con el identificador de tipo de servicio 02, mientras que se le marca el subtipo 08. Este telemando se envía cuando se quieren adquirir datos de dispositivos físicos concretos. La nave responde con un informe utilizando TM[2,9]. Los campos en los que se almacenan estas etiquetas dentro del paquete se mostrarán en las subsecciones 2.2.1 y 2.2.2.

En la figura 2.2 se muestra la estructura genérica del paquete. El diseño fue ideado por la CCSDS. Cuando se construye un telemando o una telemetría con esta estructura, debemos fijarnos en los campos *packet secondary header* y *user data*, ya que estos son los que contienen los datos cruciales que se envían. En las subsecciones 2.2.1 y 2.2.2 se muestra la estructura interna de estos dos campos. Por otro lado, el cuarto bit, perteneciente a la bandera (o *flag*) denominada *packet type*, marca si el paquete que se está enviando es un telemando (bit a 1) o es una telemetría (bit a 0). El resto de los campos aportan un mayor contexto a la información que se quiere enviar. A destacar:

- El campo *application process ID* contiene el identificador único del proceso de aplicación a bordo, el cual puede ser el origen de la telemetría dentro de la nave o el destino del telemando.
- Las telemetrías usan el campo *packet sequence count* para incrementar secuencialmente el conteo de paquetes enviados por el proceso de aplicación de origen. En cambio, los telemandos pueden usar este campo del mismo modo, o como *packet name*, cuya idea es transportar un identificador único para ayudar en la asociación de los telemandos.
- Por último, el campo *packet data length* almacena la longitud del *packet data field* (número de bytes).

Tabla 2.1: Descripción de cada uno de los servicios PUS[8]

Servicio PUS	Título	Descripción
		Continúa en la página siguiente

Tabla 2.1 – continúa en la página anterior

Servicio PUS	Título	Descripción
ST[01]	verificación de solicitud	concierne a cada proceso de aplicación que participa en el enrutamiento de solicitudes a los procesos de aplicación responsables de su ejecución, y para cada solicitud, el proceso de aplicación responsable de su ejecución, es decir, el proceso de aplicación que aloja el servicio que ejecuta la solicitud
ST[02]	acceso al dispositivo	proporciona la capacidad de distribuir comandos a los dispositivos integrados y adquirir datos de ellos
ST[03]	gestión interna (*housekeeping)*	proporciona los medios para controlar y adaptar el plan de informes de la nave espacial según las fases de la misión
ST[04]	Informe de estadísticas de parámetros	proporciona la capacidad de evaluar estadísticas a bordo para una lista de parámetros a bordo
ST[05]	Informes de eventos	proporciona la capacidad de informar información de importancia operativa que no se proporciona explícitamente en los informes iniciados por el proveedor de otro servicio
ST[06]	gestión de memoria	proporciona la capacidad de cargar, volcar y comprobar el contenido de estas memorias sin excluir que la misma memoria sea utilizada por más de un servicio de gestión de memoria o que se utilicen memorias superpuestas a bordo
ST[07]	reservado	-
ST[08]	gestión de funciones	proporciona la capacidad de realizar funciones específicas de un proceso de aplicación
ST[09]	gestión del tiempo	proporciona funcionalidades relacionadas con la generación de informes de tiempo
ST[10]	reservado	-
ST[11]	programación basada en el tiempo	proporciona la capacidad de controlar los procesos de las aplicaciones a bordo mediante solicitudes precargadas a bordo de la nave espacial y liberadas en el momento oportuno
ST[12]	monitoreo a bordo	proporciona la capacidad de monitorizar parámetros o grupos de parámetros a bordo y reaccionar ante las infracciones de las condiciones de monitorización relacionadas mediante la generación de eventos. Los informes de eventos resultantes pueden enviarse a tierra y ser recibidos a bordo, por ejemplo, por un subservicio de acción de eventos

Continúa en la página siguiente

Tabla 2.1 – continúa en la página anterior

Servicio PUS	Título	Descripción
ST[13]	transferencia de paquetes grandes	implementa un protocolo de separación de aquellos paquetes grandes en pequeós, cada uno de ellos contiene una parte del paquete grande
ST[14]	control de reenvío en tiempo real	proporciona la capacidad de controlar el envío a tierra de informes (informes de verificación, respuestas y datos) generados por los servicios a bordo. Los informes se envían a tierra dentro de un canal de telemetría en tiempo real
ST[15]	almacenamiento y recuperación a bordo	proporciona la capacidad de seleccionar informes generados por otros servicios a bordo y almacenarlos en almacenes de paquetes, y de permitir que el sistema terrestre gestione los informes en los almacenes de paquetes y solicite su enlace descendente
ST[16]	reservado	-
ST[17]	prueba	proporciona la capacidad de activar las funciones de prueba implementadas a bordo y de informar los resultados de dichas pruebas
ST[18]	procedimiento de control a bordo	cumple con el estándar de procedimientos de control a bordo de la nave espacial y lo complementa (consulte *ECSS-E-ST-70-01C*[10])
ST[19]	evento-acción (*event-action*)	proporciona la capacidad de definir acciones a bordo que pueden ejecutarse de forma autónoma cuando ocurren eventos específicos a bordo
ST[20]	gestión de parámetros	proporciona capacidades para gestionar los parámetros a bordo, incluyendo la lectura de valores actuales, el establecimiento de nuevos valores y la redefinición de ubicaciones y propiedades de los parámetros
ST[21]	secuenciación de solicitudes	proporciona la capacidad de gestionar la liberación de una secuencia de solicitudes a bordo. También proporciona capacidades para la carga, el control y la generación de informes de secuencias a bordo
ST[22]	programación basada en puestos	proporciona la capacidad de controlar los procesos de las aplicaciones a bordo mediante solicitudes precargadas en la nave espacial y liberadas cuando la nave espacial alcanza la posición asociada en la órbita
ST[23]	gestión de archivos	proporciona la capacidad de administrar los sistemas de archivos y archivos a bordo

TC packet PUS version number	acknowledgement flags	message type ID		source ID	spare
		service type ID	message subtype ID		
enumerated (4 bits)	enumerated (4 bits)	enumerated (8 bits)	enumerated (8 bits)	enumerated (16 bits)	fixed-size bit-string
					optional

Figura 2.3: Estructura interna del campo *packet secondary header* de un telemando[8]

2.2.1 Telemando

Un telemando (también denominado telecontrol) en el contexto espacial es el envío de indicaciones a distancia mediante el empleo de una señal de radiofrecuencia. Sirve para dar órdenes precisas al sistema remoto al que apuntemos. La tasa de datos de telemandos puede llegar a ser del orden de kbps o bps. La palabra telecontrol contiene dos raíces: *tele*, procedente del griego, que significa distancia; y *control*, del verbo controlar. Frecuentemente se emplea el término telecomando como sustitutivo de telemando o telecontros. Sin embargo, este término no lo reconoce la Real Academia Española (RAE), por lo que en este trabajo no lo utilizaremos.

El estándar de telemandos que utiliza actualmente la ESA está definido en el documento *ECSS-E-ST-70-41C* de la ECSS. Este documento muestra en sus anexos las diferentes estructuras que deberá afrontar el telemando según el tipo de servicio y subservicio que se quiera enviar.

En la sección 2.2 se mostró la figura 2.2 y se explicó que los subcampos *packet secondary header* y *user data*, pertenecientes al *packet data*, integran los datos enviados por los telemandos y telemetrías. En la figura 2.3 se muestra el interior del campo *packet secondary header*. Como se puede comprobar, este también se divide en campos. La información aquí contenida ayuda a dar contexto del tipo de paquete que se está leyendo:

- El campo *acknowledgement flags* permite activar los tipos de telemetría a recibir:
  - TM[1,1]: genera telemetrías de aceptación.
  - TM[1,3]: genera telemetrías de inicio de ejecución.
  - TM[1,5]: genera telemetrías de progreso de ejecución.
  - TM[1,7]: genera telemetrías de fin de ejecución.
- *Service type ID* identifica el tipo de servicio PUS que solicita el telemando.
- *Message subtype ID* almacena el subtipo concreto que se quiere emplear.
- *Source ID* identifica el origen del telemando. Puede proceder de un dispositivo en concreto dentro de la nave espacial.

Para ver el contenido del campo *user data* debemos fijarnos en la figura 2.4. Aquí se encuentra el peso importante de los datos:

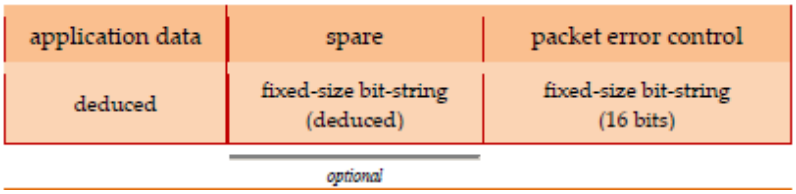


Figura 2.4: Estructura interna del campo *user data* de un telemando[8]

- El *application data* contiene los datos específicos de cada telemando enviados por el segmento terrestre (centro de control, terminales remotos, equipo postlanzamiento, estación terrestre, etc). Cada subtipo de servicio PUS contiene una serie de datos específicos concretos. Siguiendo el ejemplo de la sección 2.2, el campo *application data* del telemando TC[2,8] contiene un entero para calcular el número de dispositivos físicos de los que se quieren adquirir los datos, seguido, en orden, del identificador del dispositivo físico, el protocolo de datos específico y los datos de comando (se repiten estos tres campos tantas veces como indique el entero). Sin embargo, el telemando TC[4,4] (que habilita la generación de informes estadísticos periódicos de parámetros) solo lleva consigo una variable de tipo tiempo, la cual sirve para ordenar el intervalo de reportes estadísticos.
- El campo *packet error control* permite detectar la integridad del telemando ante cualquier pérdida o corrupción de los datos.

2.2.2 Telemetría

La telemetría es el contrapunto al telemando: es el envío de información hacia el operador del sistema. Dicha información contiene la medición remota de magnitudes físicas o sirve para dar respuesta a los telemandos recibidos con órdenes concretas. La tasa de datos de las telemetrías se encuentra entre kbps y Mbps. El término también contiene dos raíces: *tele* y *metron* (que significa medida), ambas procedentes del griego. Tal como ocurrió con los telemandos, se introdujo la estructura de telemetría actual en el estándar *ECSS-E-ST-70-41C*. La figura 2.5 contiene los campos internos del *packet secondary header*. A continuación, se nombran las diferencias con respecto al telemando:

- Para saber si el tiempo de referencia de la nave espacial se encuentra sincronizado o desincronizado, se utiliza el campo *spacecraft time reference status*.
- *Subservice type* y *message subtype* funcionan igual que en los telemandos: identifican el tipo y subtipo de servicio PUS que se envía.
- En el campo *destination ID* se guarda el identificador del destino al que debe llegar la telemetría.
- El campo *time* sirve para almacenar el On-Board Time (OBT) del momento en el que se crea la telemetría.

Los campos internos del *user data* de la telemetría se emplean igual que en el telemando, con la diferencia de que el nombre del campo *application data* aquí se llama *source data*. De la misma forma, los datos que transporta este campo cambian completamente dependiendo de la telemetría construida. La telemetría TM[5,4] (comparte informes con la gravedad de una incidencia detectada) lleva consigo el identificador del evento definido y los datos auxiliares. Por contra, la telemetría TM[9,3] (genera informes de tiempos Segmento de Día CCSDS (CDS)) transporta la tasa de valor exponencial, el tiempo de la nave y el estado de referencia de tiempo de la nave espacial.

TM packet PUS version number	spacecraft time reference status	message type ID		message type counter	destination ID	time	spare
		service type ID	message subtype ID				
enumerated (4 bits)	enumerated (4 bits)	enumerated (8 bits)	enumerated (8 bits)	unsigned integer (16 bits)	enumerated (16 bits)	absolute time	fixed-size bit-string
							optional

Figura 2.5: Estructura interna del campo *packet secondary header* de una telemetría[8]

source data	spare	packet error control
deduced	fixed-size bit-string (deduced)	fixed-size bit-string (16 bits)
	optional	optional

Figura 2.6: Estructura interna del campo *user data* de un telemetría[8]

2.3 TASTE

El programa TASTE supone la unión de una diversidad de herramientas libres dedicadas al desarrollo de sistemas embebidos en tiempo real. El origen proviene de la ESA como respuesta a las demandas que exigía el mercado. TASTE permite integrar de manera sencilla y heterogénea distintas piezas de código, algunas producidas manualmente (C o Ada), y otras automáticamente a partir de herramientas de modelaje externo (MATLAB Simulink y SDL a través de OpenGEODE). Además, se utiliza Abstract Syntax Notation One (ASN.1) para integrar las distintas piezas. En este proyecto se hará uso de ASN.1 y SDL, por lo que las explicaremos más adelante. El objetivo es simple: a partir de funciones heterogéneas, TASTE debe ser capaz de construir un sistema de software homogéneo, donde cada una de sus partes reconozca al resto y sepa comunicarse.[11]

La arquitectura de un proyecto se construye a partir de componentes o *functions*. Un componente representa a una parte dentro del sistema. Puede ser un instrumento, un sensor, el cerebro de la nave o una función concreta dentro de otro componente. Cada uno de estos componentes tiene su propia implementación. En su configuración se puede elegir las implementaciones por cada uno de los componentes. Estas implementaciones pueden ser editadas para introducir nuevas funcionalidades: se puede introducir nuevos métodos de código si se trata de una implementación de C o Ada, se le puede integrar un nuevo estado a la máquina a través de SDL, o se puede crear una interfaz gráfica editable a través de código en Python.

Las conexiones que se generan entre los componentes para el paso de mensajes se llaman interfaces. Dichas conexiones pueden ser manejadas desde las implementaciones GUI y SDL. TASTE proporciona las siguientes interfaces válidas:[1]

- Cíclicas - activan periódicamente el comportamiento de una función. En este tipo de interfaz actúa únicamente un componente, pues no está pensada para el envío de información. Se puede activar en la configuración el periodo deseado. Solamente disponible para funciones SDL.

- Esporádica y Protegida - permiten comunicar dos componentes. Se tratan de canales asíncronos o síncronos con colas entre funciones. Se puede configurar el tipo de parámetro ASN.1 a enviar, la

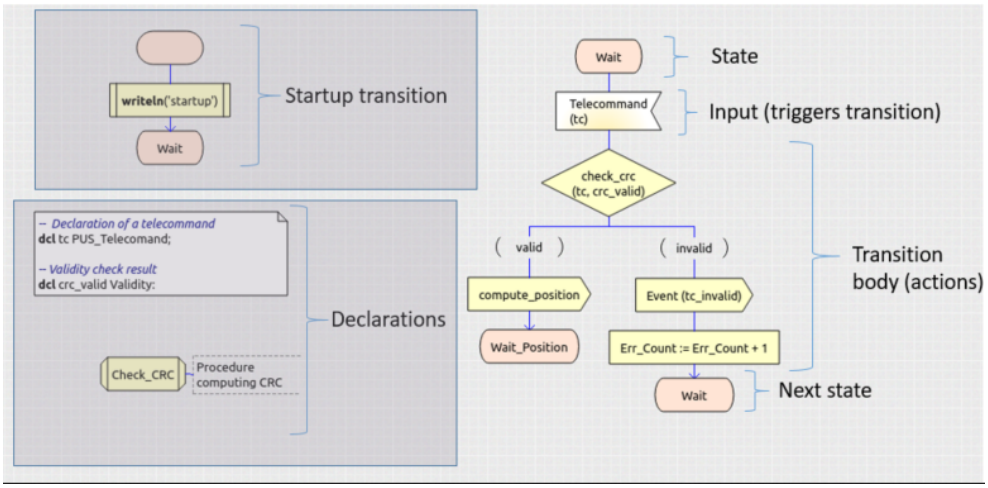


Figura 2.7: Diagrama básico de transición SDL[14]

frecuencia máxima con la que se proporciona el comportamiento asociado a la señal y el tamaño de la cola.

2.3.1 SDL

SDL es un lenguaje de modelado intuitivo para la creación de máquinas de estado. La construcción del modelo se implementa de manera visual, aunque cuenta con una sintaxis textual. Su notación gráfica permite construir modelos mediante editores interactivos. Su estructura capta, de manera sencilla, el comportamiento de sistemas embebidos en tiempo real. EL estándar fue establecido por la Unión Internacional de Telecomunicaciones (ITU) bajo la referencia *Z100*. [12]

Una máquina de estados es una conjunción de estados y de transiciones entre ellos. La estructura de una transición en SDL tiene la siguiente forma:

```
trigger; (condition | action)*,
```

donde el disparador o *trigger* de la transición puede ser la recepción de una señal a través de una interfaz, un evento *timeout* o una señal continua. Si la señal se recibe a través de una interfaz, esta se introduce en la cola a la espera de que el estado actual de la máquina sea leer la señal. Una señal continua se define como un predicado sobre las variables de la función y la transición se ejecuta cuando se evalúa como verdadera.[1]

El programa TASTE incluye un editor gráfico SDL de código abierto que genera código Ada, llamado OpenGEODE. Desde este editor gráfico se pueden diseñar todos los estados por los que la máquina pasa. Permite manejar temporizadores, ejecutar métodos externos y recibir o enviar comunicaciones con el resto de los componentes a través de sus interfaces. En cada uno de los estados de la máquina existe un disparador, ya sea explícito o implícito, para cada señal recibida [13]. En la figura 2.7 se muestra la construcción típica de SDL. Todos los componentes deben comenzar con una transición inicial. Se pueden crear distintas ramas por las que proceder, así como el uso de condicionales *if else*. Las llamadas a métodos externos se pueden realizar mediante llamadas a procedimientos [14].

2.3.2 ASN.1

ASN.1 es una notación utilizada para la estructura de los datos transmitidos por los mensajes intercambiados entre distintas entidades comunicantes en entornos heterogéneos. Como tal, se trata de un

estándar y se sigue actualizando. Inicialmente, fue normalizada por la Organización Internacional de Normalización (ISO), aunque ahora la ITU se encarga de desarrollarlo y mantenerlo. ASN.1 es capaz de soportar el intercambio de información de cualquier forma: desde audio hasta vídeo y datos en crudo.

En el contexto de este trabajo, la herramienta TASTE emplea este tipo de notación para el envío de mensajes entre los componentes de un sistema. Los sistemas de notaciones más conocidos son Notación de Objeto de JavaScript (JSON) y Lenguaje de Marcas Extensible (XML). Sin embargo, ASN.1 siempre ha sido una solución más natural debido a su sencillez de comprensión. Proporciona los medios para describir mensajes, tanto abstractos como físicos. EL usuario escribe el constructor en formato ASN.1 y el compilador integrado es capaz de generar una representación equivalente de este tipo de dato en el lenguaje de implementación elegido por el usuario (C, SDL, Ada, etc).[15]

Se ha generado la tabla 2.2 para mostrar algunos de los tipos de datos que se pueden usar. También se ha añadido el tipo de dato en C que lo sustituye. ASN.1 cuenta con un repertorio más rico que el de C, por lo que, en algunos casos, es necesario definir manualmente algunas estructuras especiales para su traducción.[16]

Tabla 2.2: Algunos tipos de datos en ASN.1

	Tipo	**Descripción**	**Equivalencia en C**
INTEGER	Primitivo	Representa números enteros	int
REAL	Primitivo	Representa números reales	float
OCTET STRING	Primitivo	Almacena una secuencia de bytes	octs
BOOLEAN	Primitivo	Almacena valores que solo pueden ser verdadero o falso	bool
NULL	Primitivo	Ausencia de valor	int
UNIVERSAL TIME	Primitivo	-	TIME
SEQUENCE	Construido	Lista ordenada de tipos de datos diferentes	struct{...}
SET	Construido	Lista no ordenada de tipos de datos diferentes	struct{...}
ENUMERATED	Construido	-	enum
CHOICE	Construido	Se almacena una lista de opciones, en la que hay que elegir uno de ellos	struct{ int choice; union {...} data }

La sintaxis utilizada para definir nuevas variables es muy parecida a la que se emplea en la mayoría de los lenguajes de programación modernos. Por ejemplo, en el código 2.1 se representa un servicio PUS definido como una secuencia de cinco tipos. Como se ha visto en la tabla 2.2, una secuencia es similar a una estructura en C. [16] Se le pueden añadir variables aunque no compartan su tipo. El nombre de la variables primitivas puede comenzar con minúscula, mientras que el tipo ha de escribirse en mayúscula. Por el contrario, el nombre de las variables construidas empieza con mayúscula.

Listado 2.1: Ejemplo de creación de alguna variables en código ASN1

```
Coche ::=  SEQUENCE {
marca OCTET STRING,
modelo INTEGER,
color OCTET STRING,
diaCompra UNIVERSAL TIME,
SET OF Concesionario )
}

Concesionario ::= SET {
nombre [O] OCTET STRING,
ciudad [1] OCTET STRING ]
}

esAmarillo ::= BOOLEAN

TipoNeumaticos ::= ENUMERATED { invierno, liso, deportivo }
```

Capítulo 3

Análisis de objetivos y metodología

Una vez concluido el capítulo dedicado al estado del arte, se enumeran todos los objetivos del proyecto. Además, se da una visión general de los pasos a seguir para la obtención de estos, se exponen los requisitos del sistema y se especifican las herramientas utilizadas para el desarrollo del proyecto.

3.1 Objetivos

En este Trabajo de Fin de Máster, el objetivo fundamental es el diseño, la implementación y la evaluación de los servicios PUS mediante un modelo generado en la herramienta TASTE. El diseño del sistema se basa en el modelo EDROOM, creado por el Space Research Group (SRG) de la Universidad de Alcalá (UAH). En la figura 3.1 se muestra gráficamente la comparativa entre el diseño antiguo y el nuevo. En la arquitectura antigua, se emplea el modelo EDROOM para generar la arquitectura del proyecto y se utiliza código externo con la idea de lanzar métodos concretos. Además, se emplea MAST para calcular la planificabilidad. En la nueva arquitectura, el programa TASTE hace el trabajo de EDROOM. Se seguirá haciendo uso del código externo para la llamada a ciertas funciones.

El objetivo primordial se disemina en un grupo de submetas a alcanzar:

- **O1**: analizar el estado del arte para el empleo de TASTE en proyectos reales.
- **O2**: realizar un modelo en TASTE que aglutine los servicios PUS, siguiendo la estructura del modelo EDROOM.
- **O3**: creación del modelo mediante el patrón general de diseño Unidad de Control de un Instrumento (ICU), dividido en una serie de subobjetivos:
  - **O3.1**: completar el código otorgado por los profesores de la asignatura SCTRSE.
  - **O3.2**: establecer los componentes que participan en el modelo, junto a sus estados correspondientes.
  - **O3.3**: enlazar los componentes mediante canales de comunicación (envío/recepción de comandos específicos internos).
  - **O3.4**: implementación del código en C++ que ejecute cada uno de los estados de los componentes.
- **O4**: validar que se cumplen las prioridades asignadas a cada uno de los actuadores del modelo.

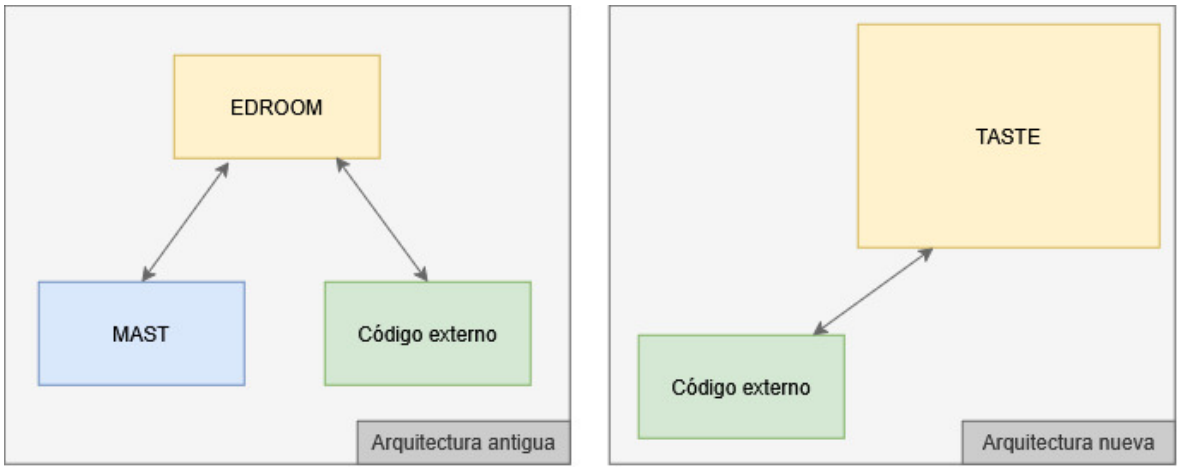


Figura 3.1: Comparación de la arquitectura antigua y nueva

- **O5**: confirmar que se cumplan los procedimientos de intercambio de telemandos y telemetrías.
- **O6**: documentar los pasos seguidos y la evaluación final del modelo para poder ser utilizado en futuros cursos por los profesores de la asignatura Software de Control en Tiempo Real para Sistemas Espaciales (SCTRSE).

3.2 Metodología de trabajo

Teniendo en cuenta los objetivos enumerados previamente, a continuación se lista la metodología de trabajo, paso a paso:

1. Formación inicial (3 meses):
   - Preparación de la máquina virtual.
   - Formación en TASTE. Se realizarán proyectos de introducción para el manejo de competencias mínimas de la herramienta. Deberemos asegurarnos que los proyectos compilan y pueden ejecutarse, generando la salida correcta.
   - Formación en servicios PUS. Aquí se incluye aquella formación dada por los profesores de la asignatura SCTRSE del Máster Universitario en Ciencia y Tecnología desde el Espacio, así como del aprendizaje autónomo.
   - Formación en la herramienta EDROOM, creada por el SRG, de la UAH. Esta enseñanza se hará a través de documentación otorgada por los profesores de la asignatura, de la misma universidad.
   - Consulta bibliográfica.
2. Diseño, implementación y evaluación del entorno generado (2 meses):
   - Definición del modelo. Se analizan los servicios PUS principales que deberán integrarse primero.
   - Implementación del sistema. Creación del modelo utilizado en las prácticas, junto a los servicios PUS elegidos.
   - Evaluación del entorno. Realizar envíos de telemandos y telemetrías para comprobar el correcto intercambio de mensajes, así como el correcto funcionamiento del sistema en su totalidad.

3. Documentación de la memoria del trabajo (0,5 meses).
4. Preparación para la defensa del trabajo (0,25 meses).

3.3 Requisitos del sistema

Para la implementación del sistema final en TASTE, se deben cumplir una serie de requisitos previos para su puesta en marcha:

- **R1**: uso de herramientas de código abierto.
- **R2**: manipulación de máquinas virtuales.
- **R3**: entender el funcionamiento de ASN.1, SDL, Ada, telemandos, telemetría y servicios PUS.
- **R4**: conocimiento en el manejo de gestores de compilación C/C++ gcc, cmake y make.
- **R5**: el sistema final debe ser escalable para futuras ampliaciones que pueda sufrir.

3.4 Herramientas y recursos

Las herramientas necesarias para la elaboración del proyecto han sido:

- Ordenador donde se lanzará la máquina virtual. No se especifica ningún Sistema Operativo (SO) en particular. En el caso de estudio, se hará uso de Windows 11 con un procesador 11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz, con 16GB de memoria RAM.
- Programa para lanzar máquinas virtuales. En el caso de estudio, se ejecutará Oracle VM VirtualBox.
- Imagen ISO para la máquina virtual. Se ejecutará el sistema operativo Ubuntu 22.04.
- Instalación del programa TASTE.
- Apuntes otorgados por los profesores de la asignatura Software de Control en Tiempo Real para Sistemas Espaciales, del Máster Universitario en Ciencia y Tecnología desde el Espacio, de la Universidad de Alcalá. Se necesitarán para abordar la implementación de los servicios PUS en el programa TASTE y EDROOM.
- Compilador C/C++ gcc.
- Herramienta cmake.
- Gestor de compilaciones make.

Capítulo 4

Desarrollo

En este capítulo se diseminará el trabajo llevado a cabo, poniendo especial atención al diseño e implementación de cada uno de los componentes que conforman el sistema final. Se pormenorizará la arquitectura del proyecto y se describirá cada uno de sus elementos: sus funciones, sus conexiones con el resto de los componentes y sus cambios de estado. Se hará uso de ilustraciones y se mostrarán partes del código para apoyar su comprensión. También se estudiará el envío del telemando hasta la posterior recepción y análisis de la situación, así como la generación de la telemetría correspondiente.

4.1 Arquitectura del proyecto

La arquitectura del proyecto está formada por la conjunción de cada una de las partes del proyecto. Para que el administrador de telemandos pueda crear la respuesta adecuada, debe producirse la inyección de los telemandos. En general, podemos dividir el sistema en cuatro fases: inyección del telemando, análisis del telemando, ejecución de la acción o evento correspondiente al tipo de comando recibido y envío de una respuesta en forma de telemetría.

La figura 4.1 muestra el diseño de la arquitectura realizada para el proyecto. En la figura aparecen todos los componentes existentes en el sistema: el administrador de telemandos "TCManager"(contiene el bucle *poll* que revisa la llegada de telemandos), el administrador de la gestión interna "HKFDIRMng"(encierra el disparador *trigger* que crea los informes de *housekeeping*), el ejecutor de telemandos en segundo plano "BKGTCExec"(como el nombre indica, se encarga de ejecutar los telemandos que le llegan) y "Legacy", el elemento al que se le delegan las llamadas internas. Más adelante, se hablará en profundidad de cada uno de los componentes y sus acciones pertinentes.

En TASTE, los componentes se pueden crear a través de una interfaz gráfica, sin necesidad de escribir el código a mano. La vista captura el modelo de la arquitectura lógica del sistema, independientemente de la implementación del software. El lenguaje Architecture Analysis & Design Language (AADL) se utiliza como formato textual intermedio para la visualización de interfaces. Esta característica es de gran utilidad, pues nos permite centrarnos en la creación de los elementos del proyecto y cambiar sus propiedades sin mayor dificultad. El código de la interfaz se genera una vez que se guardan los cambios realizados. Durante este trabajo, se va a utilizar la palabra componente para nombrar los elementos del sistema. Sin embargo, la palabra técnica de la que hace uso el programa es *function*. Existen varias formas de funciones:

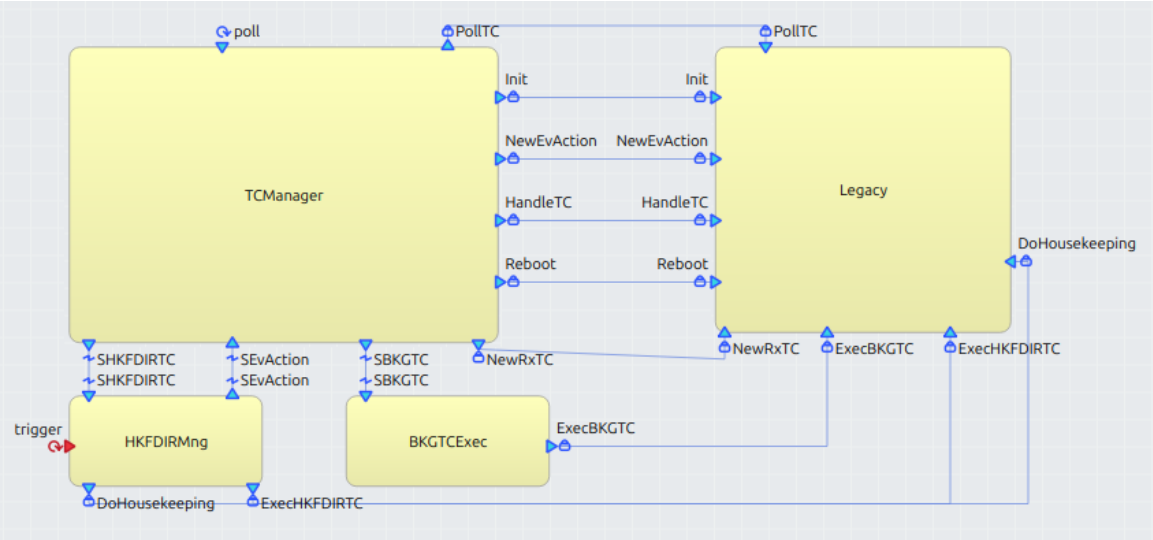


Figura 4.1: Arquitectura principal del proyecto

- *Functions* que representan un componente activo del sistema (una máquina de estados o una ley de control), es decir, algo capaz de comunicarse con las *functions* circundantes. Los componentes de este proyecto se comportan de esta manera.
- *Nested functions* que permiten estructurar el sistema de forma jerárquica (agrupando por contexto o para mejorar la legibilidad). El anidamiento es recursivo.
- Tipos de *function* que contienen un comportamiento genérico y pueden instanciarse.
- Instancias de *functions*.

Otra de las fortalezas que tiene TASTE es que permite establecer prioridades a cada uno de estos. Se ha marcado con una mayor prioridad a TCManager, ya que por este componente pasan el resto de comunicaciones y ocurren los procesos principales. HKFDIRMng tiene la segunda mayor prioridad, pues es quien genera los informes de *housekeeping* y de Detección, Aislamiento y Recuperación de Fallos (FDIR). Legacy cuenta con la tercera marca de prioridad y BKGTCExec con la última.

Dentro de la configuración de una *function*, (ver imagen 4.2), se pueden establecer distintos detalles. Se puede elegir desde aquí el número de instancias mínimas y máximas del componente, así como la instancia de la que deriva. También se puede comprobar cómo la prioridad de inicialización se encuentra en 1. En la pestaña "*Implementations*"se añaden o eliminan las implementaciones (C/C++, SDL, etc) del componente. Cada uno de los componentes puede contener varias implementaciones. Leer la sección 4.6 para más información.

Aunque en este capítulo se va a ahondar en los componentes TCManager, HKFDIRMng y BKGTCExec en sus respectivas secciones, es necesario explicar primero el motivo de existencia del cuarto componente. El modelo EDROOM presenta una ventaja en diseño con respecto a TASTE. En el diseño de la interfaz, es posible incluir guardas y llamadas a procedimientos en los componentes. Estos dos instrumentos permiten ejecutar funciones de código que se conectan al código externo. En otras palabras, cuando un componente cambia de estado, puede necesitar hacer la llamada a una pila de datos o activar un instrumento. EDROOM se encarga de poner a disposición una llamada al procedimiento que se ejecutará al realizar el cambio de estado. Sin embargo, el programa TASTE no goza de esa funcionalidad. Es por ello que se decidió crear el componente Legacy: su única función es mantener el contexto del sistema una vez que arranca y ejecutar las llamadas a procedimiento del resto de componentes.

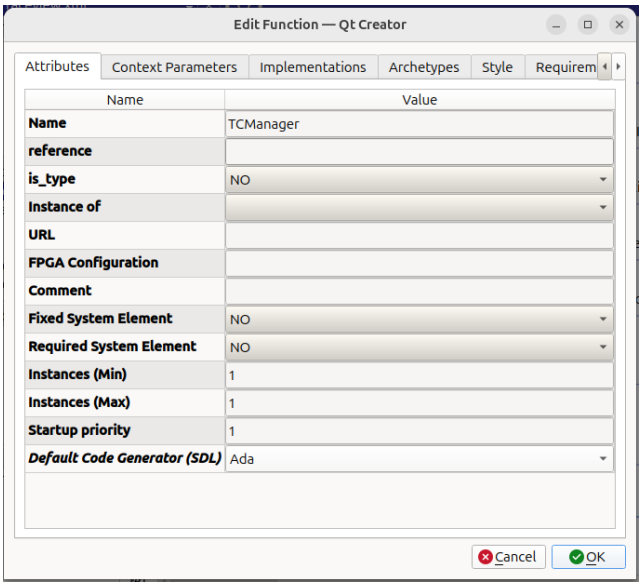


Figura 4.2: Ventana de configuración de un componente o *function*

En su nivel más abstracto, un sistema modelado con TASTE contiene elementos que intercambian mensajes con otros elementos a través de interfaces. La semántica de la vista de interfaces deriva en gran medida de la vista de comunicación de SDL. Las interfaces, como las comunicaciones, pueden ser:

- Periódicas, en cuyo caso no se le añade ningún parámetro. Se usa en aquellos casos en los que se quiere repetir cíclicamente una actividad. Las interfaces *poll* y *trigger* son de este tipo. A cada una se le ha imputado un periodo distinto, siendo 100 ms y 1000 ms, respectivamente.

- Esporádicas. Representan las comunicaciones asíncronas y no es obligatorio el envío de parámetros. Una comunicación asíncrona es aquel intercambio de mensajes entre un emisor y un receptor que no coinciden en el tiempo. Es decir, el emisor transmite la información sin tener la certeza del momento en el que el nodo opuesto va a leer la información, y el receptor lee el mensaje cuando accede al canal. Esto requiere que el canal cuente con un *buffer* de almacenamiento de mensajes. Si se llega a su límite y llega un nuevo mensaje, este se pierde al no haber manera de almacenarlo. Las interfaces de este tipo están marcadas con una virgulilla y se utilizan en la comunicación entre los componentes TCManager, HKFDIRMng y BKGTCExec.

- Síncronas. Engloba el tipo contrario de comunicación. Una comunicación síncrona ocurre cuando el emisor y el receptor coinciden en el tiempo. Esto significa que el emisor se queda a la espera de que el receptor lea el mensaje una vez que lo envía por el canal. A su vez, el receptor se queda leyendo el canal hasta que se produce la llegada del paquete. En TASTE las interfaces síncronas tienen un funcionamiento similar al de las llamadas de funciones de C. La llamada es inmediata y bloqueante: hasta que no termina el intercambio de datos, no se cierra.
  Al igual que con el resto de interfaces, aquí no es obligatorio el envío de parámetros. Para el manejo de la exclusión mutua, existe la opción de proteger la interfaz. La exclusión mutua se utiliza en la programación concurrente para evitar que más de un proceso (siendo el emisor y receptor dos procesos) entren en la sección crítica del código (aquella que modifica datos de la memoria). En el desarrollo de este trabajo, se ha decidido proteger las interfaces de Legacy con el resto de componentes. Así, es posible asegurarse de que Legacy realice inmediatamente la llamada a las funciones que el resto de los componentes necesitan.

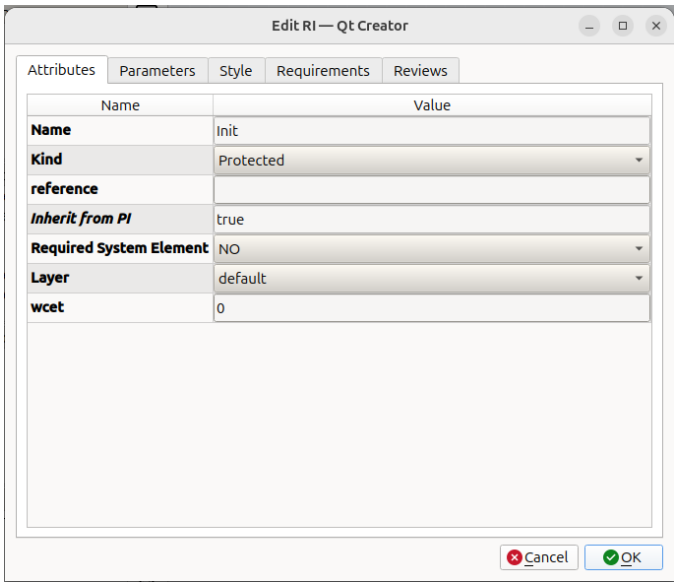


Figura 4.3: Ventana de configuración de una interfaz

En la configuración de una interfaz, se puede elegir el tipo de comunicación a establecer entre los componentes (figura 4.3. Cuando una interfaz es cíclica, permite imputar el periodo de repetición de su acción. Si la interfaz es asíncrona, se podrá elegir la capacidad máxima del *buffer* del canal. En el caso de que la comunicación necesite parámetros, estos se establecen en la pestaña "*Parameters*". Los parámetros son variables que pueden ser de distintos tipos y permiten, en el caso del proyecto, comunicar datos de los comandos o sobre las decisiones que se toman. A cada parámetro se le asigna un nombre, el tipo de dato del lenguaje y la dirección (*in* o *out*).

Para entender mejor la manera en que se comunican las distintas herramientas de nuestro proyecto, se han dibujado varios diagramas de secuencia que muestran los posibles escenarios a los que puede verse avocado el sistema cuando recibe un telemando. Se detalla el proceso que completa desde el punto de vista de TCManager y HKFDIRMng y sus interfaces cíclicas. Como se ha comentado anteriormente, el componente TCManager cuenta con la interfaz *poll*. Esta interfaz se activa cada 100 ms y su objetivo final es que el sistema analice y verifique si ha llegado algún telemando y es válido (su construcción es correcta y contiene un tipo y subtipo reconocidos por el estándar). Los pasos se detallan a continuación:

1. La aparición de la terminal se produce una vez que se ejecuta el programa y no desaparece hasta que se aborte, por lo que permite enviar varios telemandos por sesión.

2. El administrador de telemandos se activa. Pasan 100 ms y activa la llamada al procedimiento *PollTC* a Legacy.

3. Legacy comprueba el OBT actual e inyecta los telemandos según el script de prueba que se quiera validar. Los scripts llevan escritas varias macros, donde se indican los datos del telemando: su tipo, subtipo, el segundo en el que se inyecta y los parámetros que necesita el comando para ejecutarse. Si hay una macro a inyectar que cuenta con el mismo valor de OBT actualizado, avisa al emulador para que lo construya.
   En el espacio, el tiempo en las naves se mide mediante el OBT, precisos relojes internos y sistemas de sincronización que las naves espaciales utilizan para registrar datos, programar operaciones y comunicarse con la Tierra. El proyecto implementa un contador de 32 bits de segundos completos, inicializado en 100000. Cada vez que se avanza un segundo, una llamada interna aumenta el valor inicial para simular el incremento de segundos desde el arranque de la ejecución, sin conversión

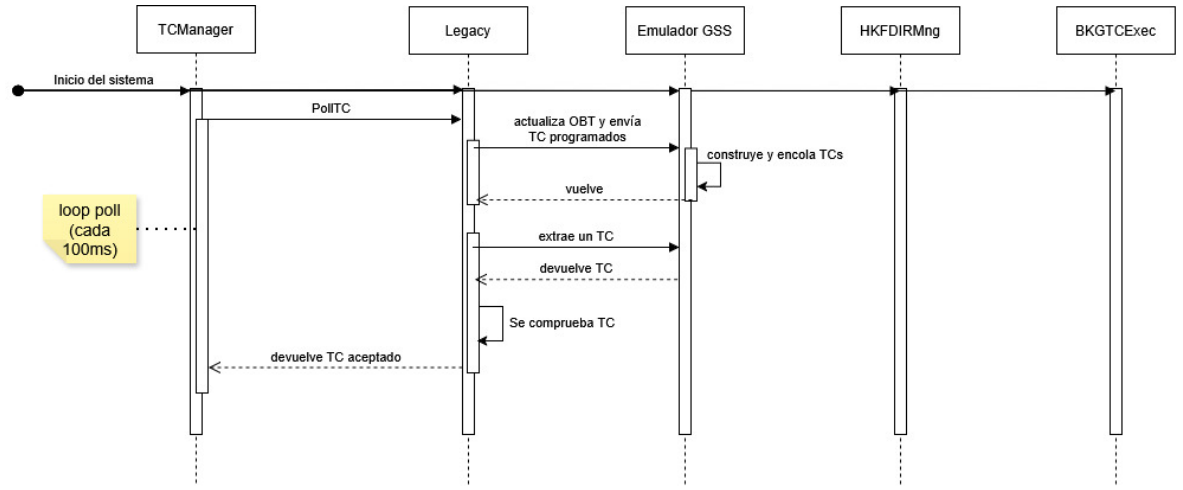


Figura 4.4: Diagrama de secuencia general de la interfaz *poll*

a UTC. Se utiliza un reloj monotónico, por lo que cambiar la fecha del ordenador no altera el OBT. El sondeo está configurado para realizarse cada 100 ms, pero el OBT avanza en intervalos de un segundo. Los sondeos retrasados se recuperan a través del bucle; las actualizaciones se omiten temporalmente mientras permanece pendiente un telemando aceptado.

4. El emulador es parte del código externo. Se encarga de mantener el tiempo de vuelo actualizado, de construir las pruebas, los telemandos y las telemetrías. Cuando recibe la señal de Legacy, construye y encola el telemando.

5. Legacy extrae el telemando entregado por el emulador y se asegura de que ha sido construido correctamente.

6. Si el telemando es válido, Legacy avisa al administrador que puede ejecutarlo.

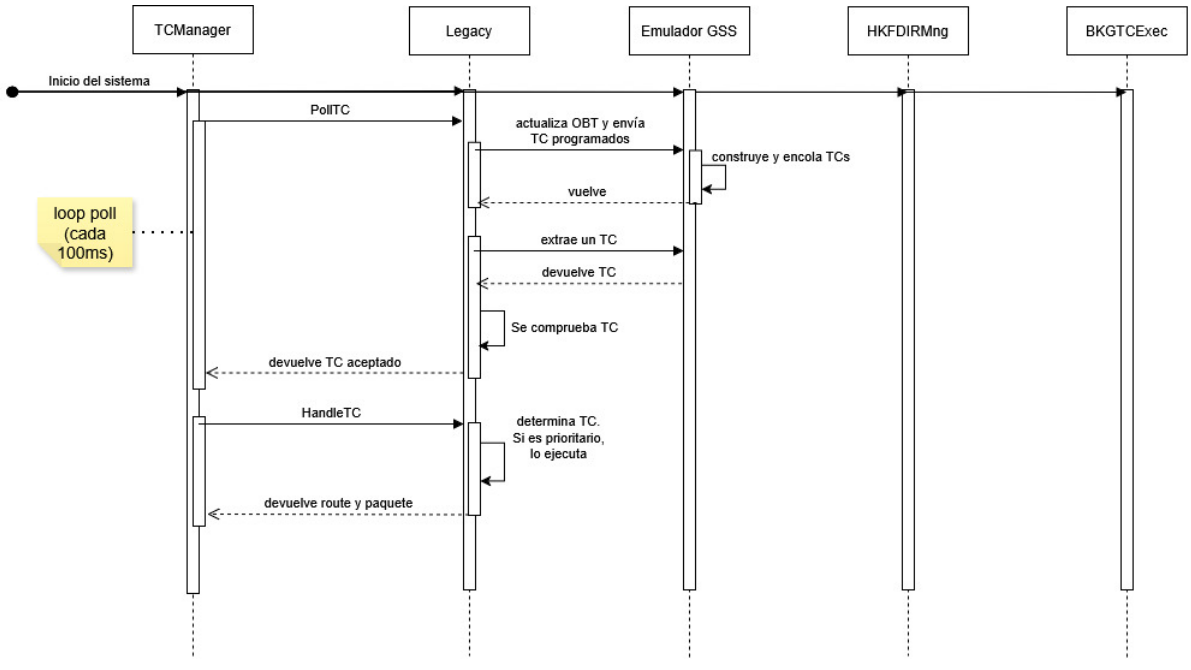


Figura 4.5: Diagrama de secuencia en caso de que el telemando sea prioritario

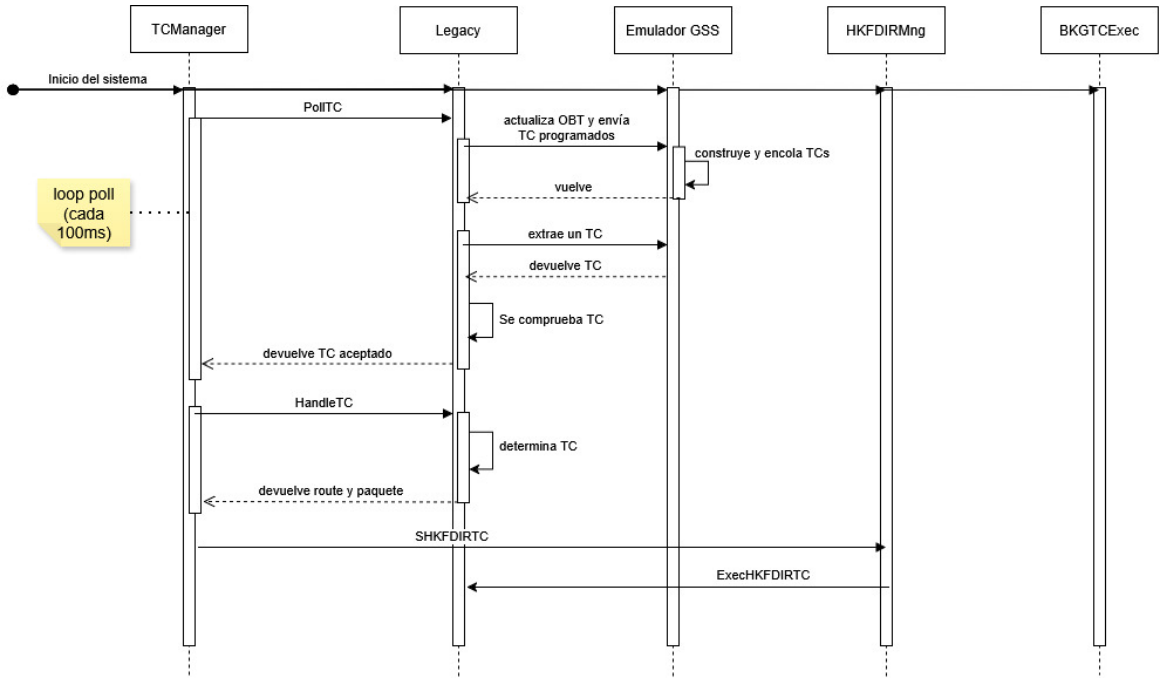


Figura 4.6: Diagrama de en caso de que el telemando provenga de *housekeeping* o FDIR

A la hora de analizar el tipo de comando, el sistema puede tratarlo de manera diferente, dependiendo de si es prioritario, si puede ejecutarse en segundo plano, si proviene de HKFDIRMng o si hay que reiniciar el sistema. Aquí entran en juego cuatro escenarios distintos:

- Se entiende como prioritario aquellos comandos que deben ser ejecutados de manera directa por el propio sistema. Su ejecución se considera esencial. En el capítulo 5 se explicarán todos los tipos de servicios PUS que se van a probar, pues no todos forman parte de la validación. Del pequeño grupo de servicios desarrollados, se han considerado los servicios 02 y 17 como prioritarios.
  En la figura 4.5 aparece el siguiente paso del diagrama y lo que ocurre si el telemando es prioritario. TCManager realiza la llamada al procedimiento *HandleTC* sobre Legacy. Este, analiza el telemando. En caso de ser prioritario lo ejecuta directamente y devuelve el resultado (una señal indicando que el comando era prioritario).

- Aunque se explicará HKFDIRMng más adelante, se puede adelantar que dicho componente se encarga de administrar el *housekeeping* (tareas rutinarias de mantenimiento de un sistema). El administrador de telemandos delegará las acciones relacionadas con los servicios de gestión interna en él (servicios 03, 05, 12 y 19).
  Cuando Legacy determina que el telemando es de gestión interna, avisa a TCManager (ver figura 4.6) a través del canal asíncrono *SHKFDIRTC*. En este momento, HKFDIRMng lo analiza y realiza la llamada al procedimiento *ExecHKFDIRTC* en dirección Legacy, el cual se encarga de ejecutarlo.

- El servicio 20 ha sido marcado como el menos prioritario, por lo que se le ha asignado a BKGTCExec para que lo administre en segundo plano. Al igual que en el caso anterior, si se determina que el telemando es de baja prioridad, TCManager se lo envía a BKGTCExec utilizando el canal asíncrono *SBKGTC* (imagen 4.7. Posteriormente, BKGTCExec lo despacha mandándolo a Legacy por el canal síncrono y protegido *ExecBKGTC*.

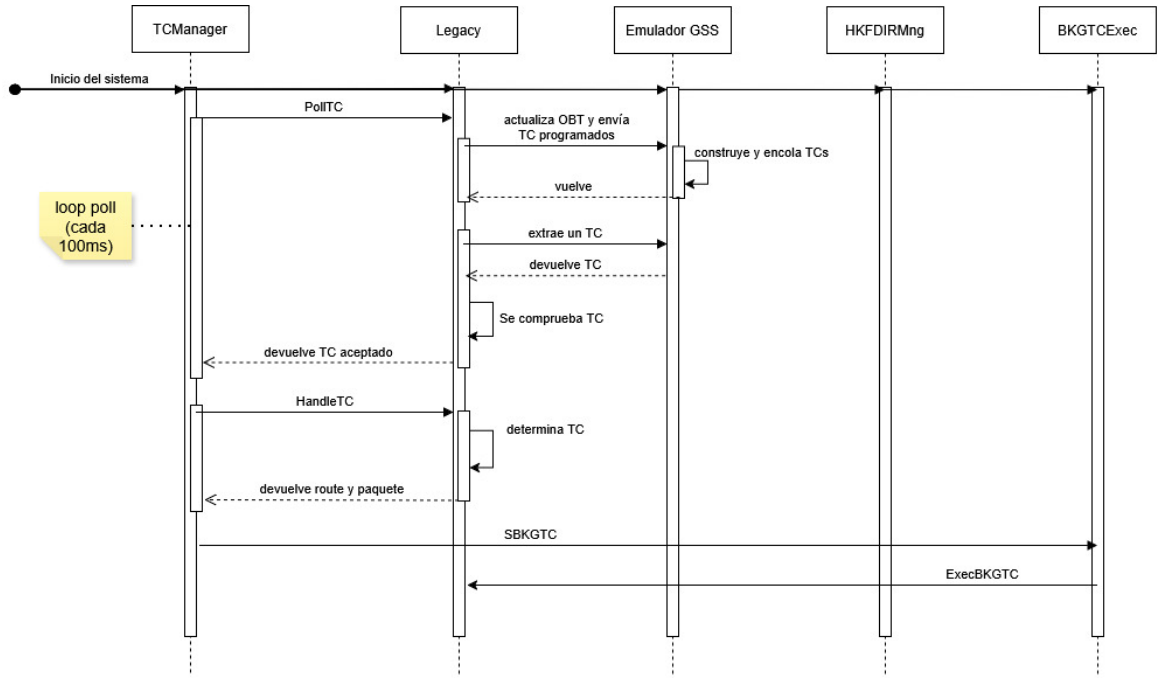


Figura 4.7: Diagrama de secuencia en caso de que el telemando se vaya a ejecutar en segundo plano

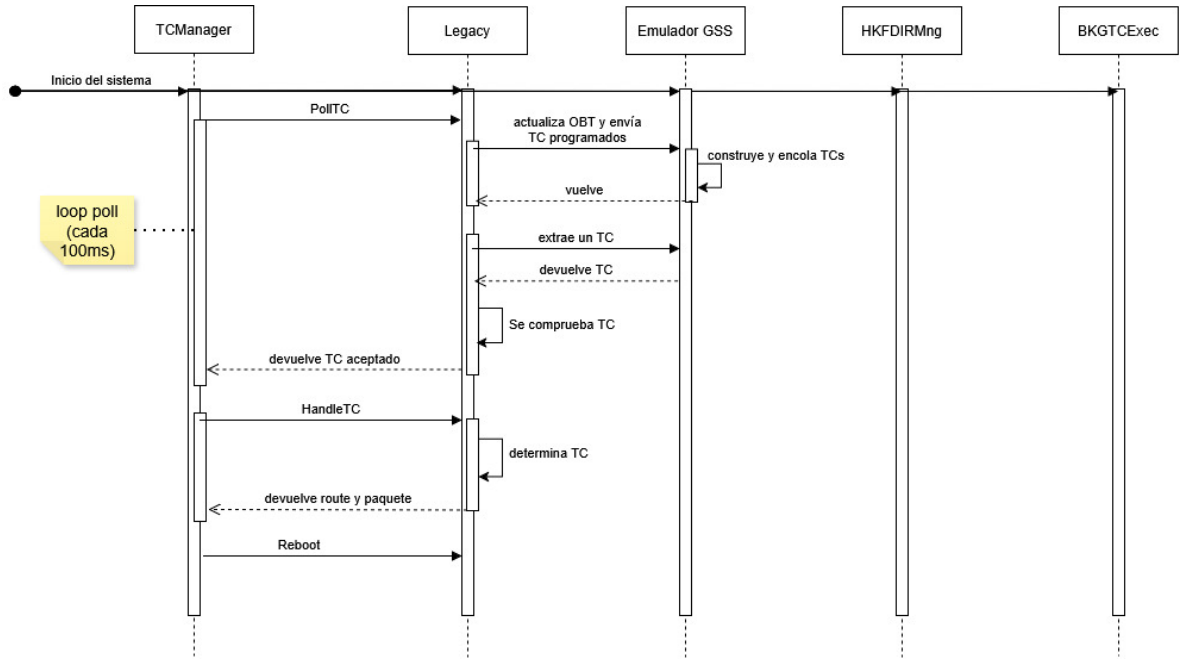


Figura 4.8: Diagrama de secuencia en caso de que el telemando de que se haya que realizar un reinicio

- Finalmente, si Legacy considera que el telemando ha de reiniciar el sistema, TCManager se lo devuelve para que comience con el reinicio (figura 4.8). Este escenario se ha ideado en caso de que llegue algún comando correspondiente a un servicio no nombrado anteriormente.

Tal como se mencionó, HKFDIRMng también cuenta con una interfaz cíclica, llamada *trigger*. Este disparador se activa cada segundo y su propósito es generar informes internos del sistema. Se ha generado el diagrama de secuencia 4.9 para analizar los movimientos entre componentes. Cuando se activa el disparador, HKFDIRMng realiza la llamada al procedimiento *DoHousekeeping* a Legacy. Este actualiza los parámetros del *System Data Pool*, genera los informes y evalúa FDIR.

Algunos de los servicios que engloba HKFDIRMng se encargan, no solo de generar informes mediante telemetrías, sino que también pueden llegar a construir telemandos. Estos son enviados a TCManager (mediante *SEvAction* para que los administre). Al tratarse de un telemando que recibe a través de un canal asíncrono, no necesita verificar si es válido, simplemente avisa a Legacy de que se ha producido un nuevo evento-acción (*NewAction*) y que determine qué hacer con él (*HandleTC*). En este punto se repite el proceso de los anteriores diagramas de secuencia.

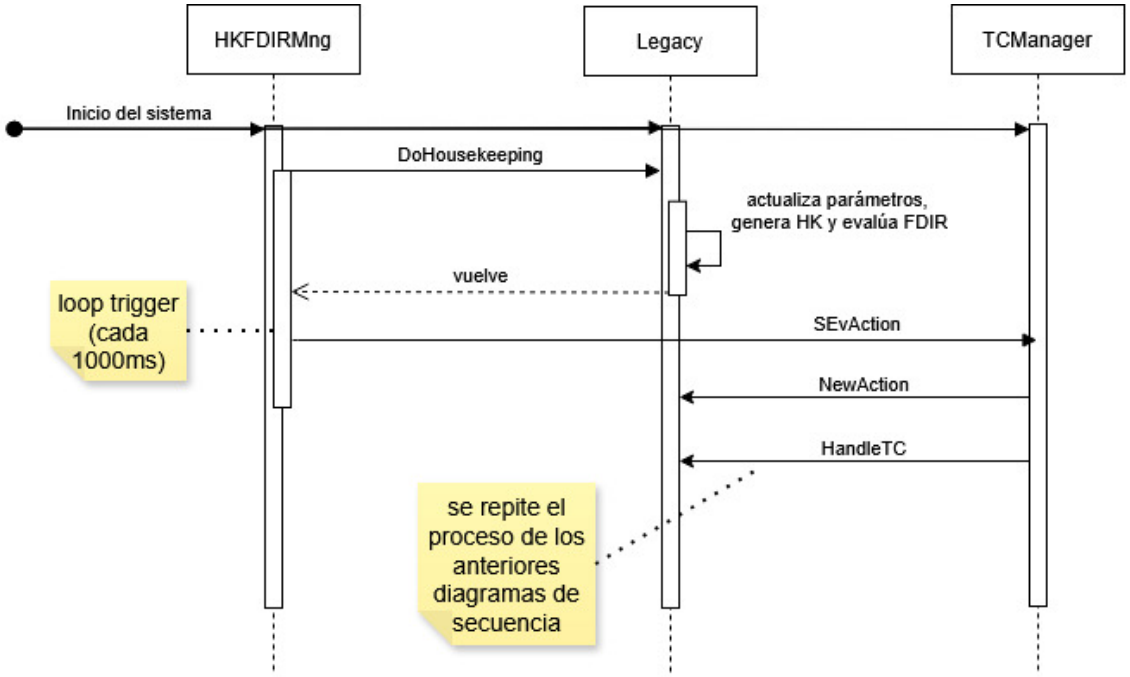


Figura 4.9: Diagrama de secuencia general de la interfaz *trigger*

4.2 Inyección de los telemandos

En el diseño original del modelo en EDROOM, la llegada de los telemandos no se hacía a través de ningún componente, sino que se producía un evento externo en forma de señal eléctrica enviada al procesador. Una vez que el procesador recibe esta señal, se produce una interrupción en el flujo y se encarga de gestionar la solicitud. En este caso, la interrupción supone la recepción de un telemando debido a la aparición del evento externo.

Listado 4.1: La función *EmuGSS_SendProgrammedTCs()* recupera los telemandos inyectados

```
void EmuGSS_SendProgrammedTCs() {
  uint32_t currentOBT = EmuHwTimeCodeGetCurrentOBT(); //Obtiene el OBT (OnBoard Time) actual
  bool tc_programmed=false;

  //El método SetCurrentTCPrograned devuelve true si hay TCs programados para el CurrentOBT
  while (EmuGSS_TCProgram::SetCurrentTCProgramed(currentOBT)) {
    //Si se llega aquí es que hay TC programado para el CurrentOBT. Se obtiene su información
        para que aparezca por consola.
    printf("\nGSS Tx TC[%i,%i] ",
        (EmuGSS_TCProgram::sCurrentTCProgram)->GetType(),
        (EmuGSS_TCProgram::sCurrentTCProgram)->GetSubtype());

    printf("%s",EmuGSS_TCProgram::sCurrentTCProgram->GetBrief());
    printf("\n");

    tc_mem_descriptor_t tcDescriptor; //Ahora se construye el descriptor del TC

    //Se obtiene el TC programado
    EmuGSS_TCProgram *ptc_program = EmuGSS_TCProgram::GetCurrentTCProgramed();

    //Se obtiene memoria para que el TC se gestione en los componentes
    tcDescriptor.p_tc_bytes = TCQueue_GetTailTCMemory();

    ptc_program->BuildTC(tcDescriptor); //Con esa memoria se construye el tcDescriptor

        TCQueue_TailTCCompleted(tcDescriptor.tc_num_bytes); //Lo añade al buffer
        tc_programmed=true;
  }
    if(tc_programmed){
        //Dispara una IRQ para avisar al gestor de TC
    CCTCManager::EDROOMEventIRQ18.SignalFromTask();
  }
}
```

Cuando el componente Legacy recibe la llamada al procedimiento *PollTC*, consulta con el emulador los telemandos a inyectar por medio de la función *EmuGSS_SendProgrammedTCs()*. Después, la función busca si hay algún telemando programado para el segundo actual. Si devuelve verdadero, se introduce en el bucle para tratar cada uno de los telemandos pendientes. Aquí, imprime por pantalla el tipo y subtipo del telemando y una pequeña descripción de su función. En el capítulo siguiente se podrán ver distintos ejemplos de comandos que muestra el sistema por la terminal. Luego, se construye el descriptor para el telemando, se recupera y se le otorga memoria para poder construir su descriptor. Finalmente, lo añade a la cola o *buffer* para que Legacy lo pueda recuperar y se provoca una Solicitud de Interrupción (IRQ) para avisar al administrador de telemandos. IRQ es una señal que envía un periférico, temporizador o módulo del sistema para avisar al procesador de que debe prestar atención a un proceso ahora mismo. La Unidad Central de Procesamiento (CPU) deja lo que está haciendo, atiende la interrupción y luego vuelve a la tarea.

El emulador, además de controlar el tiempo de vuelo y de inyectar los telemandos, también se encarga de imprimir por pantalla la telemetría. La técnica es similar a los telemandos: recupera el tipo y subtipo de la telemetría. Sabiendo ambos datos, es capaz de determinar la información que necesita para construir el paquete.

4.3 Administrador de telemandos

La pieza angular de la arquitectura es el administrador de telemandos, o TCManager. Se trata del cerebro de la máquina. Es el encargado de recibir los telemandos que se le inyectan, gestionar el resto de los componentes, recibir la telemetría de *housekeeping* y ejecutar los comandos o cambios que se requieran por los telemandos. También muestra en pantalla las telemetrías que se generan.

En la figura 4.10 se dibujan los estados pertenecientes al administrador, junto con la secuencia de acciones que le hacen cambiar entre estos. El administrador cuenta con cuatro estados (es el componente con más estados): inicial, *ready*, *validtc* y *reboot*. Para entender cómo el administrador se mueve entre distintos estados, se muestra el proceso interno que ocurre en el componente en la figura 4.11. El diagrama de estados y las llamadas a los procedimientos se ha realizado en SDL.

Todo componente comienza en un estado inicial, marcado con el óvalo rosa vacío. Cuando se compila y ejecuta el proyecto, se activan cada uno de los componentes. En el caso del administrador, antes de pasar al estado *Ready*, se realiza la llamada al procedimiento *doInit*, el cual llama a la función *Init* del componente *Legacy*. Esta función se encarga de arrancar los servicios PUS para el objeto RxTC (ver código 4.2).

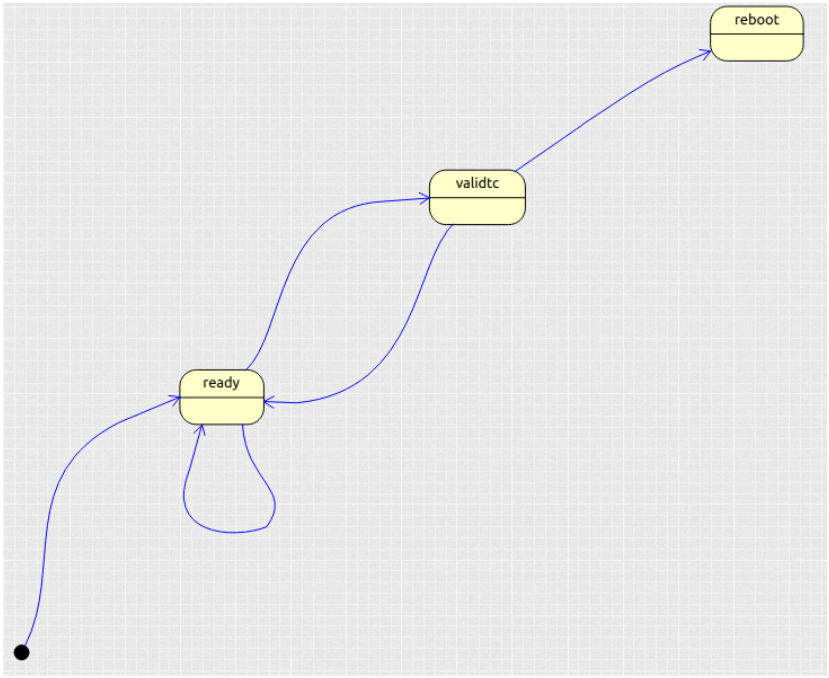


Figura 4.10: Diagrama de estados del componente TCManager

Listado 4.2: Inicialización de TCManager

```
1  extern "C" void FInit(void)
2  {
3  ...
4      CCTCManager::RxTC.MaskIRQ();
5      CCTCManager::RxTC.InstallHandler();
6      pus_services_startup(&CCTCManager::RxTC);
7      CCTCManager::RxTC.UnMaskIRQ();
8  ...
9  void legacy_PI_Init(void) { FInit(); }
10 }
```

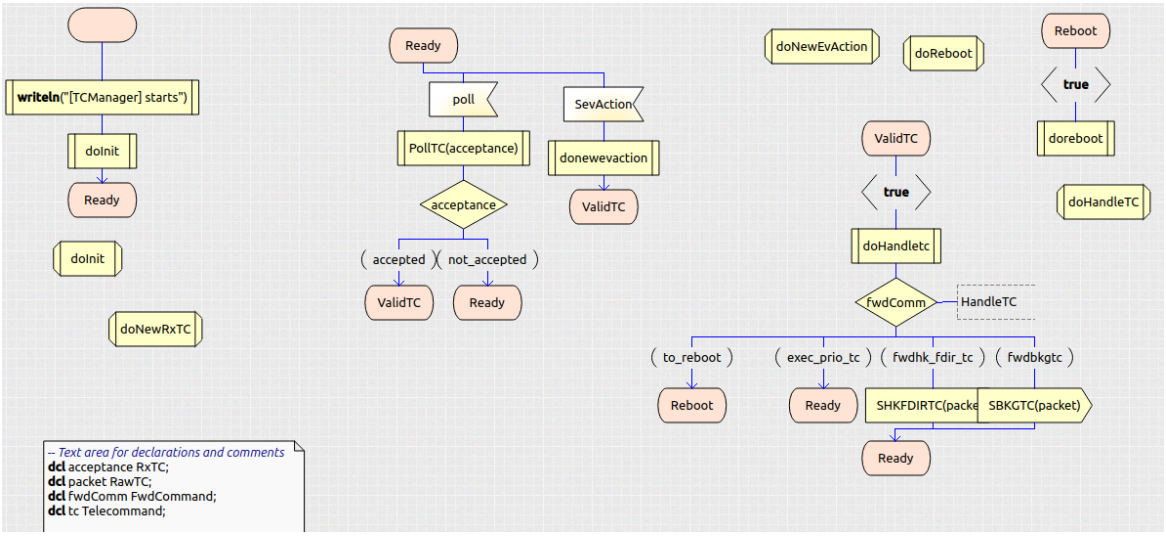


Figura 4.11: Proceso interno del componente TCManager en SDL

Una vez que entra en el estado *Ready*, el administrador se queda a la espera de que le llegue alguno de los siguientes mensajes:

- Por un lado, el disparador **poll** se encarga de revisar periódicamente si hay un nuevo telemando disponible. Si se activa, hace la llamada al procedimiento de la función *PollTC(acceptance)* (código 4.3). En esta función, el componente Legacy lee la cola de telemandos buscando algún telemando mediante *EmuGSS_SendProgrammedTCs()*. Si encuentra uno, intenta evaluarlo llamando a la función *ValidPacket(bytes, size)* y a través de *DoAcceptation()* . Si el paquete es aceptado, se lanza la función *FMngTCAcceptation()*, se marca como tal y se devuelve a TCManager el resultado. En caso contrario, se ejecuta *FMngTCRejection()*.
  Es importante aquí explicar lo siguiente: cuando se hace la llamada al procedimiento, se envía como parámetro una variable ASN.1 de tipo enumerado, llamada *acceptance*. Esta variable puede adoptar dos valores: *accepted* o *not_accepted*. De esta manera, cuando se acepta el telemando, se marca el valor de esta variable con el valor de *accepted*. Así, el proceso interno de TCManager, una vez acaba la llamada al procedimiento y tiene que tomar una decisión sobre la variable (símbolo en forma de rombo amarillo), viajará al estado **ValidTC** si el telemando era correcto o se quedará en el mismo estado en caso contrario.

- Por otro lado, el componente TCManager también puede recibir mensajes del componente HKFDIRMng a través de la interfaz *SevAction*. Esto significa que se ha realizado un evento-acción perteneciente al servicio PUS 19 y requiere ser tratado por TCManager. Un servicio de evento-acción es un mecanismo automatizado a bordo que activa acciones predefinidas de la nave espacial siempre que ocurre un evento específico del sistema. Permite que un satélite o nave espacial reaccione instantáneamente ante situaciones rutinarias o anómalas sin esperar a recibir un comando de ida y vuelta desde el control de tierra.
  TCManager realiza la llamada al procedimiento *NewEvAction* del componente Legacy. Esta función se encarga únicamente de extraer el evento-acción y de aceptar el mensaje. La diferencia con **pool** radica en que, en este caso, no se valida si el mensaje se acepta o no, simplemente se viaja al estado **ValidTC**.

Listado 4.3: Leer la cola de mensajes de TCManager

```
void legacy_PI_PollTC(asn1SccRxTC *accepted)
{
    ...
    EmuGSS_SendProgrammedTCs();
    if (TCQueue_IsEmpty()) return;  //La cola está vacía
    uint16_t size = 0;
    const auto *bytes = TCQueue_GetHeadTCMemory(size);
    if (!ValidPacket(bytes, size)) {
        TCQueue_HeadTCExtracted();
        return;
    }
    ...
    ctxt_legacy.VAcceptReport = ctxt_legacy.VCurrentTC.DoAcceptation();
    if (GAcceptTC()) {  //Si se acepta el paquete
        FMngTCAcceptation();
        ctxt_legacy.has_accepted_tc = true;
        *accepted = asn1SccRxTC_accepted;
    } else {
        FMngTCRejection();
    }
}
```

En el estado **ValidTC**, llega el momento de gestionar el comando incluido en el telemando recibido. TCManager realiza la llamada al procedimiento *HandleTC* de Legacy con el objetivo de que determine la etiqueta que se le va a otorgar (código 4.4). Para ello, cuenta con unas funciones "guardas"para cada tipo de etiqueta (prioritario, FDIR o *housekeeping*, segundo plano o reinicio). Cada una de ellas revisa el tipo de servicio PUS que almacena el telemando y devuelve verdadero si tiene su etiqueta. Legacy va llamando a cada una de las guardas hasta que alguna devuelva verdadero y poder ejecutar la función correspondiente. Por ejemplo, si el telemando contiene el tipo 17, quiere decir que pertenece al servicio 17. Legacy comienza a llamar a las guardas hasta que llegar al final, momento en el que no le queda otra que llamar a *FExecPrioTC()*. Esta función simplemente ejecuta el comando. En cambio, si el servicio ha sido etiquetado como FDIR, de baja importancia o de reinicio del sistema, Legacy manda la señal de vuelta a TCManager.

Terminada la llamada *HandleTC*, TCManager vuelve a tener el control de la situación. Según la señal devuelta por Legacy, TCManager tomará un camino distinto. En orden de aparición de la imagen 4.11, si la señal viene marcada como reinicio, TCManager viaja al estado **Reboot** y realiza la llamada al procedimiento *Reboot* de Legacy para que comience el reinicio del sistema. Si se marca como prioritario, vuelve al estado **Ready** (en este caso no hace nada más, pues se explicó en los diagramas de secuencia que Legacy ejecuta el telemando directamente). Si la señal avisa que el telemando es FDIR o *housekeeping*, TCManager se lo envía a HKFDIRMng a través del canal asíncrono SHKFDIRTC para que se encargue de su tratamiento. En el caso de los telemandos con servicio 20, marcados como poco prioritarios, su destino es ir a BKGTCExec. En estos dos últimos casos, después de enviar el paquete con el comando, TCManager vuelve al estado **Ready**.

El traspaso de señales y paquetes se hace mediante variables ASN.1. Estas variables hay que declararlas en una nota de texto dentro de SDL (ver esquina inferior izquierda de la imagen 4.11). Legacy apunta el valor *accepted* o *not_accepted* en la variable *acceptance* de tipo *RxTC*. Así, TCManager sabe qué camino escoger. Lo mismo ocurre con la variable *fwdComm* de tipo *FwdCommand*. Cuando TCManager tiene que enviar el paquete con el telemando a HKFDIRMng o BKGTCExec, lo envuelve dentro de la

variable *packet* de tipo *RawTC*. Al final del capítulo se listarán todas las variables ASN.1 generadas para el proyecto.

Listado 4.4: Gestión del tipo de comando por parte de TCManager y Legacy

```
void legacy_PI_HandleTC(asn1SccFwdCommand *route, asn1SccRawTC *packet)
{
    *route = asn1SccFwdCommand_exec_prio_tc;
    *packet = {};
    if (!ctxt_legacy.has_accepted_tc) return;
    FTCExecCtrl();
    if (GFwdToHK_FDIR()) {
        FFwdHK_FDIRTC(packet);
        *route = asn1SccFwdCommand_fwdhk_fdir_tc;
    } else if (GFwdToBKG()) {
        FFwdBKGTC(packet);
        *route = asn1SccFwdCommand_fwdbkgtc;
    } else if (GToReboot()) {
        FExecRebootTC();
        *route = asn1SccFwdCommand_to_reboot;
    } else {
        FExecPrioTC();
    }
    ctxt_legacy.VCurrentTC.Reset();
    ctxt_legacy.has_accepted_tc = false;
}
```

4.4 HKFDIRMng

Que una nave espacial sea capaz de responder y actuar ante las incidencias internas es de vital importancia. El componente HKFDIRMng se encarga de la gestión interna de la nave. Se entiende como gestión interna la generación de informes de *housekeeping* sobre los parámetros del *System Data Pool* (servicio PUS 03), que expresen la gravedad de las anomalías que se detecten (servicio 05), además de permitir reaccionar ante el incumplimiento de las condiciones de monitorización asociadas a la generación de eventos (servicio 12) y proporcionar la capacidad de definir acciones a bordo que pueden ejecutarse de forma autónoma cuando se producen eventos específicos a bordo (evento-acción del servicio 19).

El tipo de servicio de *housekeeping* (mantenimiento y supervisión del estado del sistema) proporciona visibilidad sobre cualquier parámetro de a bordo integrado en estructuras de informe de parámetros de *housekeeping* o de diagnóstico, según lo requiera la misión. Las estructuras de informe de parámetros utilizadas por este servicio pueden estar predefinidas a bordo o crearse según sea necesario.

Los informes de parámetros, tanto de mantenimiento interno como de diagnóstico, pueden generarse periódicamente o a petición. Es posible habilitar o deshabilitar la generación periódica de cada tipo de informe de parámetros. Por ejemplo, se puede desactivar la generación periódica de informes correspondientes a un tipo de informe de parámetros de mantenimiento interno para reducir la carga de procesamiento a bordo. Asimismo, se puede habilitar un tipo de informe de parámetros de diagnóstico cuando se produzca una anomalía concreta y desactivarlo en otros momentos.

HKFDIRMng cuenta únicamente con dos estados: el inicial y el estado **Ready**. Al igual que TCManager, el estado **Ready** reacciona ante la llegada de mensajes a través de dos canales:

- El disparador *trigger* se activa cada segundo. Cuando se produce dicha activación, HKFDIRMng lee el canal y realiza la llamada al procedimiento *DoHousekeeping* de Legacy. Esta función es muy simple, su único propósito es el de actualizar los parámetros del *System Data Pool* y de activar el

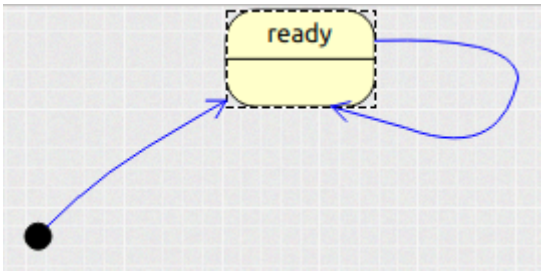


Figura 4.12: Diagrama de estados del componente HKFDIRMng

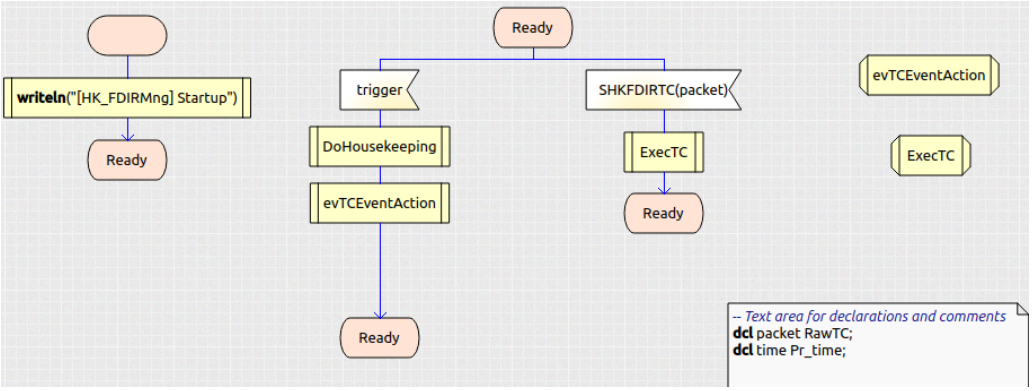


Figura 4.13: Proceso interno del componente HKFDIRMng en SDL

housekeeping o FDIR, dependiendo del tipo de prueba a ejecutar. Actualmente, el sistema no es capaz administrar ambas situaciones, por lo que se debe activar el *housekeeping* cuando se quiera hacer la validación del servicio 03, o activar FDIR en el caso de los servicios 05, 12 y 19.
El *System Data Pool* es el conjunto de parámetros representativos de un sistema que pueden medirse o calcularse, como la temperatura de un sensor o el grado de inclinación de un panel solar. La monitorización de estos parámetros es diferencial, pues son capaces de indicarnos si hay algún problema en la nave. Por ejemplo, si no se detecta que la temperatura del procesador ha subido drásticamente, se puede llegar a perder el control de la nave. Cada parámetro del *System Data Pool* tiene un identificador único asignado que se denomina Parameter IDentifier (PID).
Cuando TCManager vuelve a tener el control de la situación, puede enviar algún telemando que haya generado por el canal *SEvAction* a TCManager para que lo administre. Esta situación es muy concreta, pues solamente el servicio 19 es capaz (hasta el momento) de generarlos.

Tabla 4.1: Algunos posibles parámetros del *System Data Pool*

PID	
0	Voltage_5v
1	Voltage_3_3v
2	Current_5v
3	Current_3_3v
4	Temp_1
5	Temp_2
...	...
N-1	CPU_usage

- La otra opción es recibir un paquete de TCManager a través del canal *SHKFDIRTC*. Dicho paquete (variable ASN.1 de tipo *RawTC*) contiene un telemando que puede pertenecer a cualquiera de los cuatro telemandos que administra HKFDIRMng. Una vez le llega el paquete y lo lee, hace la llamada al procedimiento *ExecHKFDIRTC* para ejecutarlo desde Legacy.

4.5 BKGTCExec

BKGTCExec es el elemento más simple del proyecto. Este componente tiene el mismo diagrama de estados que HKFDIRMng (fig 4.12). En su estado **Ready**, se queda leyendo el canal *SBKGTC* a la espera de la llegada de paquetes (variables ASN.1 de tipo *RawTC*) con comandos del servicio 20. Una vez que llega uno, lleva a cabo la llamada al procedimiento *ExecBKGTC* de Legacy, el cual ejecuta el comando (ver figura 4.14).

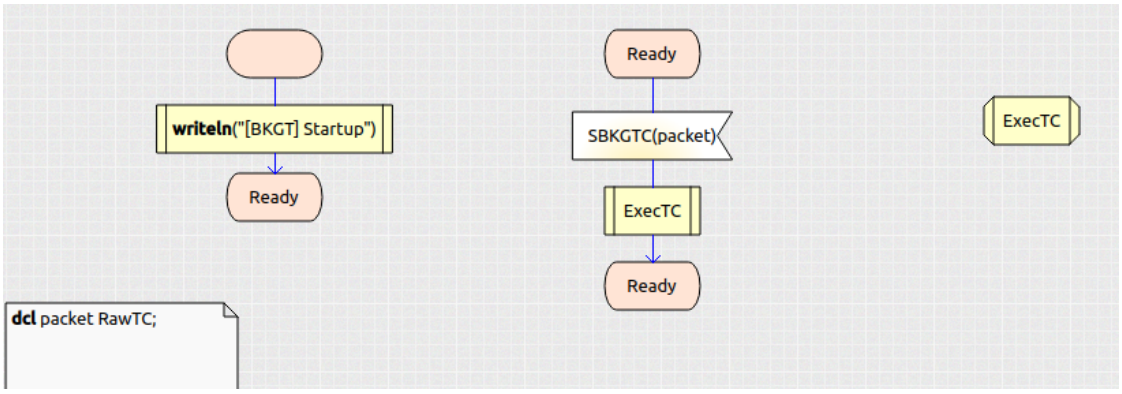


Figura 4.14: Proceso interno del componente BKGTCExec

4.6 Variables ASN.1 del proyecto

Se ha querido mostrar la generación de las variables ASN.1 que utilizan los componentes del sistema. *RawTC* es una cadena lo suficientemente larga como para poder almacenar toda la información de un telemando. *RxTC* y *FwdCommand* son enumerados que engloban las diferentes opciones que puede adoptar el recorrido de validación y gestión del telemando.

Listado 4.5: Variables generadas en ASN.1

```
1  SCTRE-DATAVIEW DEFINITIONS ::=
2  BEGIN
3
4  -- Complete PUS packet copied by value across asynchronous interfaces.
5  RawTC ::= OCTET STRING (SIZE (0..256))
6
7  RxTC  ::= ENUMERATED { not-accepted, accepted }
8  FwdCommand  ::= ENUMERATED { to-reboot, exec-prio-tc, fwdhk-fdir-tc, fwdbkgtc }
9
10 END
```

Capítulo 5

Resultados

Este capítulo está dedicado íntegramente al análisis de los resultados obtenidos en las pruebas realizadas sobre la arquitectura diseñada y desgranada en el anterior capítulo, con el objetivo de evaluar su rendimiento y su eficacia para afrontar las situaciones a las que se someterá (objetivos **O4** y **O5**). Para ello, se inyectarán telemandos en el sistema. Este deberá ser capaz de responder con las telemetrías correctas.

En las secciones siguientes, se detallará el flujo de telemandos que contendrá cada prueba y se mostrará la respuesta que se genera en cada caso. Para saber si el contenido es correcto, el contenido de los archivos que servirán como pruebas será el mismo que en el sistema montado en clase, por lo que la salida por pantalla deberá ser exactamente la misma.

5.1 Validación de los servicios PUS 01 y 17

Comenzaremos realizando la validación del servicio PUS 17. Este servicio se utiliza comúnmente para comprobar que se obtiene una conexión con el dispositivo. En informática, este tipo de pruebas se conocen como *ping* y se realizan al principio de la conexión. Se busca obtener respuesta del dispositivo con el que queremos comunicarnos. Una respuesta positiva (o simplemente una respuesta) nos indica que la conexión es viable y que el otro dispositivo está preparado para recibir mensajes. En el caso de los servicios PUS, se envía el telemando TC[17,1] como señal *ping*. El dispositivo contrario deberá contestar con la telemetría TM[17,2], que incluye el informe del intento de conexión.

En esta subsección se ha introducido, además del 17, el servicio 01. Este servicio aparecerá en el resto de validaciones, pues es intrínseco a todo intercambio de mensajes. Este servicio ayuda a verificar las distintas etapas de ejecución de un telemando: desde su aceptación hasta la entrega del informe de enrutamiento, pasando por el inicio, progreso y finalización de la ejecución del telemando. Todas estas acciones tienen asociada una telemetría. A continuación, se van a enumerar todas aquellas que aparecerán en la validación:

- La telemetría TM[1,1] muestra el informe de verificación de aceptación exitosa. Es decir, el telemando se ha aceptado y va a dar comienzo la ejecución de este. Sin embargo, si resulta que hubo algún problema con el telemando, se utiliza TM[1,2].

Listado 5.1: Validación de los servicios PUS 01 y 17

```
...
#define FT_UAH_ASW_0010_TIME_step0 (OBT_AFTER_POWER_ON + 3)

EmuGSS_TCProgram17_1 prog_FT_0010_step_0(FT_UAH_ASW_0010_TIME_step0,
          "FT_UAH_ASW_SERV_17_0010 step 0, Connection Test");
...
#define FT_UAH_ASW_0020_TIME_step0 (OBT_AFTER_POWER_ON + 8)

EmuGSS_TCProgram17_1_DISABLE_1_7 prog_FT_0020_step_0(FT_UAH_ASW_0020_TIME_step0,
          "FT_UAH_ASW_SERV_1_DISABLE_1_7_0020 step 0, Disable TM[1,7]");

...
#define FT_UAH_ASW_0030_TIME_step0 (OBT_AFTER_POWER_ON + 10)

EmuGSS_TCProgram17_3 prog_FT_0030_step_0(FT_UAH_ASW_0030_TIME_step0,
    "FT_UAH_ASW_SERV_1_0030 step 0, Rejected TC",0xFF);

#define FT_UAH_ASW_0040_TIME_step0 (OBT_AFTER_POWER_ON + 12)

EmuGSS_TCProgram17_1 prog_FT_0100_step_0(FT_UAH_ASW_0040_TIME_step0,
          "FT_SOLO_EPD_TCContainment_0100 step 0");

EmuGSS_TCProgram17_1 prog_FT_0100_step_1(FT_UAH_ASW_0040_TIME_step0,
          "FT_SOLO_EPD_TCContainment_0100 step 1");

EmuGSS_TCProgram17_1 prog_FT_0100_step_2(FT_UAH_ASW_0040_TIME_step0,
          "FT_SOLO_EPD_TCContainment_0100 step 2");

EmuGSS_TCProgram17_1 prog_FT_0100_step_3(FT_UAH_ASW_0040_TIME_step0,
          "FT_SOLO_EPD_TCContainment_0100 step 3");

EmuGSS_TCProgram17_1 prog_FT_0100_step_4(FT_UAH_ASW_0040_TIME_step0,
          "FT_SOLO_EPD_TCContainment_0100 step 4");

EmuGSS_TCProgram17_1 prog_FT_0100_step_5(FT_UAH_ASW_0040_TIME_step0,
          "FT_SOLO_EPD_TCContainment_0100 step 5");

EmuGSS_TCProgram17_1 prog_FT_0100_step_6(FT_UAH_ASW_0040_TIME_step0,
          "FT_SOLO_EPD_TCContainment_0100 step 6");

EmuGSS_TCProgram17_1 prog_FT_0100_step_7(FT_UAH_ASW_0040_TIME_step0,
          "FT_SOLO_EPD_TCContainment_0100 step 7");

EmuGSS_TCProgram17_1 prog_FT_0100_step_8(FT_UAH_ASW_0040_TIME_step0,
          "FT_SOLO_EPD_TCContainment_0100 step 8");

EmuGSS_TCProgram17_1 prog_FT_0100_step_9(FT_UAH_ASW_0040_TIME_step0,
          "FT_SOLO_EPD_TCContainment_0100 step 9");
```

- Una vez que comienza la ejecución del telemando, llega la telemetría TM[1,3], la cual indica que el inicio de la ejecución ha sido exitoso. En cambio, si falla el inicio de ejecución de este, se mostrará TM[1,4].
- Por último, cuando termina la ejecución del telemando y ha sido exitosa, se envía la telemetría TM[1,7]. Por el contrario, si no se pudo completar satisfactoriamente la ejecución, se usa TM[1,8].

Las pruebas que se van a realizar en el proyecto fueron diseñadas por los profesores de la asignatura. Por cada una de las pruebas se escribió en C++ un archivo distinto. Las pruebas utilizan macros para habilitar o deshabilitar cada una de las inyecciones de telemandos. En el código 5.1, aparecen las macros habilitadas para esta validación. Las líneas que comienzan con la palabra **#define** almacenan el tiempo de envío del telemando, teniendo en cuenta que el tiempo de vuelo (OBT) de la nave comienza en el ciclo 100000. Por lo tanto, se almacena en la variable **FT__UAH__ASW__0010__TIME__step0** el segundo 100003, es decir, el segundo en el cual se lanzará el telemando.

En la línea 4 aparece el primer telemando que se va a inyectar en el sistema. El nombre de la clase que inyecta el telemando tiene la forma **EmuGSS__TCProgramXX__YY**. En este caso, se está marcando que se va a enviar el telemando TC[17,1]. El nombre del objeto **prog__FT__0010__step__0** contiene el código de identificación de la prueba (0010) y el paso o *step* (0). El parámetro paso se utiliza, en caso de que se manden varios telemandos seguidos, para diferenciarlo. Entre paréntesis se incluye el identificador de la prueba, el paso y la descripción de la prueba. Todo lo que aparezca entre paréntesis se imprimirá por pantalla junto al tipo y subtipo del telemando.

Además de los ya mencionados, también se pueden incluir parámetros únicos para darle contexto al comando. Por ejemplo, en la prueba **FT__UAH__ASW__0030__TIME__step0**, también se envía el *application process ID* del paquete, marcado con el valor **0xFF**. En el resto de pruebas se explicarán más posibilidades.

5.1.1 Resultado esperado

En la tabla 5.1 se muestra de manera clara qué se espera que ocurra cuando se inyecten los comandos. La columna **Caso** contiene el identificador de la prueba. El segundo en el que se inyecta el telemando se almacena en la columna **Tiempo** (el sistema arranca en el segundo 100000). La columna **Telemando y datos** indica el tipo y subtipo de comando que se quiere ejecutar, así como los parámetros que se añaden a la macro para ampliar el contexto.

En el segundo caso, la desactivación afecta únicamente al acuse de recibo de finalización de ese telemando. La prueba de conexión se ejecuta normalmente y los telemandos posteriores vuelven a solicitar TM[1,7].

Tabla 5.1: Telemandos y telemetrías esperados

Caso	Paso	Tiempo	Telemando y datos	Efecto esperado
0010	0	100003	TC[17,1]	aparecerá por pantalla TM[1,1], TM[1,3], TM[17,2] y TM[1,7]
0020	0	100008	TC[17,1]: desactiva TM[1,7]	aparecerá por pantalla TM[1,1], TM[1,3] y TM[17,2]
0030	0	100010	TC[17,3]	El sistema no reconoce ese telemando y envía TM[1,2]
0100	0-9	100012	Diez TC[17,1]	Cada una debe producir las en el instante siguiente las cuatro mismas telemetrías del caso 0010: cuarenta telemetrías en total

```
make -C work run
make[1]: se entra en el directorio '/home/jorge/sctre/work'
make -j -C build -f Makefile.taste run
make[2]: se entra en el directorio '/home/jorge/sctre/work/build'
# make sure the generated artefacts appear in space creator
touch ../../*.pro
# Coverage = FALSE
cd ../binaries && taste-run-and-trace ./demo || :
[-] Loaded filters
[TCManager] starts
GSS Rx TM [5,1] Informative EvID = 0x1001 : START_UP

[TASTE] Initialization completed for function TCManager
[HK_FDIRMng] Startup
[TASTE] Initialization completed for function HKFDIRMng
[TASTE] Initialization completed for function Legacy
[BKGT] Startup
[TASTE] Initialization completed for function BKGTCExec

Current OBT is = 100001
```

Figura 5.1: Inicialización del terminal al comenzar la ejecución

El objetivo de la ráfaga de telemandos es comprobar el control de tasa. El límite es de dos telemandos por segundo: cuando se alcanza, el receptor espera antes de despachar el siguiente. Sin otro tráfico, cabe esperar aproximadamente dos comandos en el ciclo 100012, dos en 100013 y así hasta 100016. El instante exacto depende de la planificación.

En condiciones normales, estos 13 telemandos generan 48 telemetrías asociadas al test: 4 + 3 + 1 + 40. El archivo programa los estímulos, pero no contiene aserciones automáticas sobre las respuestas.

5.1.2 Resultados de la validación

Antes de comenzar a analizar el resultado de la prueba por pantalla, conviene detenerse a comentar la inicialización del terminal al ejecutar el proyecto (figura 5.1). El sistema produce la telemetría TM[5,1] para informar de que está en modo arranque y preparado para usarse. El programa TASTE nos comunica cuando se produce la inicialización de los cuatro componentes (o *function*) en orden de prioridad (se cumple el objetivo **O4**). También se puede apreciar la implementación SDL de los componentes TCManager, HKFDIRMng y BKGTCExec (en las figuras 4.11, 4.13 y 4.14 se imprime por pantalla cuando se inicializa la implementación del SDL de los componentes).

Comenzamos con la validación. En la figura 5.2, se muestra por pantalla el envío de los tres primeros telemandos mostrados en 5.1. Se puede apreciar la correcta aparición del OBT por cada segundo que pasa desde el inicio de la ejecución. Los tiempos de aparición de cada uno de los telemandos coinciden con el tiempo que se les marca en la macro. Además, se imprime correctamente tanto el código del telemando como el contenido que se presenta entre paréntesis.

Seguidos de los telemandos, se encuentran las telemetrías:

- El primer telemando recibe TM[1,1], TM[1,3], TM[17,2] y TM[1,7]. El orden de aparición es consecuente con la acción que refleja cada una de las telemetrías:

  1. Primero, la llegada de TM[1,1] indica que se ha aceptado el telemando.

  2. Luego, la aparición de TM[1,3] señala que el comienzo de la ejecución del comando ha resultado exitoso.

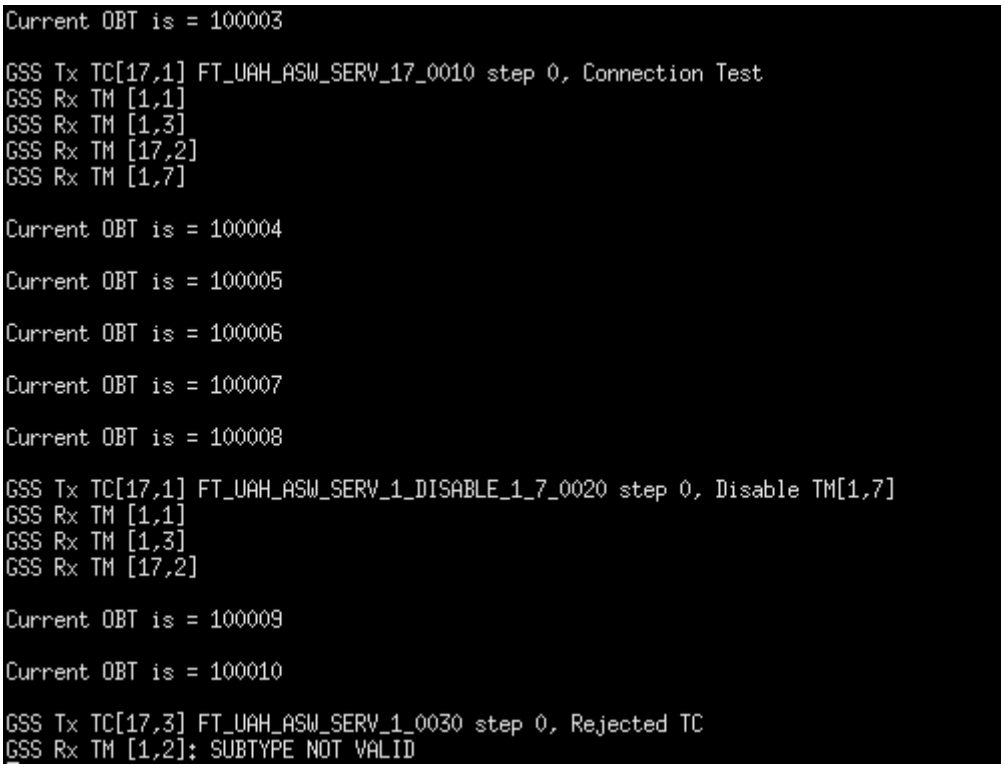


Figura 5.2: Se inyectan los telemandos TC[17,1] y TC[17,3]

3. Entre el inicio y la finalización de la ejecución del comando, se responde con la telemetría adecuada al comando que se ha hecho llegar a través del telemando. En este caso, se contesta con TM[17,2] al envío de TC[17,1].

4. Finalmente, se hace saber que la ejecución del comando ha finalizado exitosamente mediante TM[1,7].

- El segundo TC[17,1] lleva explícitamente consigo la orden de deshabilitar la aparición de TM[1,7] como parte de las telemetrías que se generan como respuesta. Se puede comprobar que la respuesta del sistema es la misma, pero no se genera la telemetría TM[1,7].

- El tercer paquete de esta imagen intenta enviar un telemando que no existe en la base actual de los servicios PUS. El sistema, al no reconocer el tipo de orden, devuelve TM[1,2], falla la verificación de aceptación, indicando que el subtipo 3 no es válido.

Los siguientes telemandos que se envían son los que aparecen a partir de la línea 20 del código 5.1. La idea es probar cómo reacciona el sistema ante la llegada masiva de comandos. Para diferenciarlos, se le indica a cada uno de ellos un número de paso o *step* distinto. En la imagen 5.3 se muestra el envío de los diez telemandos de tipo TC[17,1] en el segundo 100012, el mismo segundo que se indica en el código que deben lanzarse.

Seguido de la ristra de telemandos, aparecen las telemetrías correspondientes. Por cada uno de ellos, se ha generado TM[1,1], TM[1,3], TM[17,2] y TM[1,7]; por lo que se puede determinar que el sistema es capaz de sostener la llegada en serie de un total de diez telemandos, respectando el límite de dos telemandos ejecutados cada segundo.

```
Current OBT is = 100012

GSS Tx TC[17,1] FT_SOLO_EPD_TCContainment_0100 step 0

GSS Tx TC[17,1] FT_SOLO_EPD_TCContainment_0100 step 1

GSS Tx TC[17,1] FT_SOLO_EPD_TCContainment_0100 step 2

GSS Tx TC[17,1] FT_SOLO_EPD_TCContainment_0100 step 3

GSS Tx TC[17,1] FT_SOLO_EPD_TCContainment_0100 step 4

GSS Tx TC[17,1] FT_SOLO_EPD_TCContainment_0100 step 5

GSS Tx TC[17,1] FT_SOLO_EPD_TCContainment_0100 step 6

GSS Tx TC[17,1] FT_SOLO_EPD_TCContainment_0100 step 7

GSS Tx TC[17,1] FT_SOLO_EPD_TCContainment_0100 step 8

GSS Tx TC[17,1] FT_SOLO_EPD_TCContainment_0100 step 9
```

Figura 5.3: Se inyectan varios telemandos TC[17,1]

```
GSS Tx TC[17,1] FT_SOLO_EPD_TCContainment_0100 step 9
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [17,2]
GSS Rx TM [1,7]
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [17,2]
GSS Rx TM [1,7]

Current OBT is = 100013
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [17,2]
GSS Rx TM [1,7]
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [17,2]
GSS Rx TM [1,7]

Current OBT is = 100014
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [17,2]
GSS Rx TM [1,7]
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [17,2]
GSS Rx TM [1,7]

Current OBT is = 100015
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [17,2]
GSS Rx TM [1,7]
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [17,2]
GSS Rx TM [1,7]

Current OBT is = 100016
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [17,2]
GSS Rx TM [1,7]
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [17,2]
GSS Rx TM [1,7]
```

Figura 5.4: Telemetrías en serie que aparecen tras la inyección de telemandos de la imagen 5.3

5.2 Validación de los servicios PUS 03

Listado 5.2: Validación de los servicios PUS 03

```
...
#define OBT_START_FT_ST03 (OBT_AFTER_POWER_ON + 0)
#define FT_UAH_ASW_ICU_SERV_3_0050_TIME (OBT_START_FT_ST03 + 20)

EmuGSS_TCProgram3_6 prog_FT_0050_step_0(FT_UAH_ASW_ICU_SERV_3_0050_TIME,
    "FT_UAH_ASW_ICU_SERV_3_0050 step 0, Disable SID 0 ",0);

#define FT_UAH_ASW_ICU_SERV_3_0055_TIME (OBT_START_FT_ST03 + 30)

EmuGSS_TCProgram3_5 prog_FT_0055_step_0(FT_UAH_ASW_ICU_SERV_3_0055_TIME,
    "FT_UAH_ASW_ICU_SERV_3_0055 step 0, Enable SID 11 ",11);

#define FT_UAH_ASW_ICU_SERV_3_0060_TIME_step0 (OBT_START_FT_ST03 + 40)

EmuGSS_TCProgram3_31 prog_FT_0060_step_0(FT_UAH_ASW_ICU_SERV_3_0060_TIME_step0,
    "FT_UAH_ASW_ICU_SERV_3_0060 step 0, Update SID 10 Rate to 2",10,2);
```

La gestión interna o *housekeeping* se realiza a través del servicio PUS 03. La función de este servicio es sencilla: generar informes de parámetros del *System Data Pool*. Un Structure ID (SID) identifica una estructura de gestión interna: un conjunto de parámetros cuyos valores se transmiten mediante TM[3,25]. La configuración inicial se ha marcado en el sistema. Se han creado tres SID, con estados iniciales, intervalos y parámetros PID distintos:

- El SID 0 se ha configurado con un estado inicial **Habilitado**, su intervalo de aparición por pantalla es de cada dos segundos e incluye los siguientes parámetros: 0, 1, 2, 3, 4.
- El SID 10 contiene el estado inicial **Habilitado**, un intervalo de cuatro ciclos y se le asignan tres parámetros: 5, 6, 7.
- El único SID con estado inicial **Deshabilitado** es el 11. Se imprime por pantalla cada seis segundos (primero debe ser habilitado para mostrar los informes) y sus parámetros son: 14, 15.

El objetivo de este test es comprobar el servicio PUS 03 para la gestión interna: desactivar el informe periódico con SID 0, activar el SID 11 y cambiar el intervalo de emisión del SID 10. Si se quiere deshabilitar el envío de informes con los parámetros de un SID en concreto, se debe utilizar TC[3,6] indicando el identificador de la estructura.

TC[3,5] habilita la generación periódica de informes de parámetros de *housekeeping*. En el telemando a inyectar, se indica en su construcción el SID que se va a habilitar. Por último, TC[3,31] actualiza la periodicidad en la que se reciben los informes del SID. En la macro, hay que especificar el identificador y la nueva periodicidad (en segundos).

5.2.1 Resultado esperado

En la tabla 5.2 se muestra de manera clara que se espera que ocurra cuando se inyecten los comandos. La columna **Caso** contiene el identificador de la prueba. El segundo en el que se inyecta el telemando se almacena en la columna **Tiempo** (el sistema arranca en el segundo 100000). La columna **Telemando y**

datos indica el tipo y subtipo de comando que se quiere ejecutar, así como los parámetros que se añaden a la macro para ampliar el contexto. N=1 indica que cada telemando contiene una sola instrucción. El campo SID ocupa 16 bits; N y el intervalo ocupan 8 bits. La última columna, **Efecto esperado**, señala qué efecto va a provocar el comando en el sistema.

Tal como ocurre en la anterior validación, cada uno de estos telemandos válidos debe producir: TM[1,1], TM[1,3] y TM[1,7]. Por tanto, se esperan nueve informes de verificación del servicio 01, además de la telemetría periódica. Las telemandos no generan inmediatamente una TM[3,25]: modifican la configuración que utilizarán los siguientes ciclos de *housekeeping*.

Tabla 5.2: Telemandos y telemetrías esperados

Caso	Tiempo	Telemando y datos	Efecto esperado
0050	100020	TC[3,6]: N=1, SID=0	Desactiva la generación periódica de TM[3,25] del SID 0
0055	100030	TC[3,5]: N=1, SID=11	Activa el SID 11, que empieza a generar TM[3,25]
0060	100040	TC[3,31]: N=1, SID=10, intervalo=2	Cambia el intervalo del SID 10 de 4 a 2 ciclos: duplica su frecuencia de emisión

5.2.2 Resultados de la validación

A la hora de arrancar el sistema, se ha activado internamente el envío de informes periódicos para el SID 0 y 10. Los informes se envían a través de TM[3,25] y se puede apreciar en la imagen 5.5 el valor de los parámetros para cada SID. Si contamos el intervalo de aparición inicial, el envío de informes para el SID 0 es cada dos segundos y para el SID 10, cuatro segundos (se cumple según lo establecido anteriormente). Por otro lado, no se imprime ningún informe del SID 11, ya que está deshabilitado.

En la imagen 5.5, se puede apreciar que, veinte segundos después del arranque del sistema, se recibe TC[3,6]. El comando indica que se quiere deshabilitar los informes para el SID 0. El sistema genera correctamente la respuesta a dicho telemando, con las telemetrías TM[1,1], TM[1,3] y TM[1,7]. Pasados unos segundos, no se vislumbra la generación de informes para dicho SID.

En el segundo 100030, se lanza TC[3,5] con el objetivo de generar informes del SID 11 cada seis segundos (el valor de la periodicidad está implícito). La respuesta por parte del sistema es exitosa, ya que aparece la ristra de telemetrías TM[1,1], TM[1,3] y TM[1,7]. En la imagen 5.6 se ve el telemando TC[3,5] y en la imagen 5.7 se aprecia la aparición de informes cada seis segundos.

Para finalizar, se quiere actualizar la periodicidad de generación de informes de cuatro a dos segundos para el SID 10. La construcción de esta macro es correcta, como se aprecia en la imagen 5.7. De la misma manera, se cumple con la aparición de informes cada dos segundos, tal como se ha solicitado.

Al activar el SID 11 o cambiar el intervalo del SID 10, el código reinicia su contador a cero. La primera emisión llega tras cinco o dos llamadas de *housekeeping*, respectivamente. El instante exacto en los límites depende del orden entre la ejecución del comando y el ciclo periódico.

Cada TM[3,25] contiene el SID seguido de los valores actuales de sus parámetros, en el orden de la configuración. El test no fija valores numéricos esperados: comprueba principalmente qué SID se transmite y con qué periodicidad.

```
Current OBT is = 100016

Current OBT is = 100017
GSS Rx TM [3,25] SID 0 Param Values {1.10,2.40,3.20,4.88,-2.50}

Current OBT is = 100018

Current OBT is = 100019
GSS Rx TM [3,25] SID 0 Param Values {1.11,2.37,3.20,4.87,-2.51}
GSS Rx TM [3,25] SID 10 Param Values {0x5,0x6,0x7}

Current OBT is = 100020

GSS Tx TC[3,6] FT_UAH_ASW_ICU_SERV_3_0050 step 0, Disable SID 0
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [1,7]

Current OBT is = 100021

Current OBT is = 100022

Current OBT is = 100023
GSS Rx TM [3,25] SID 10 Param Values {0x5,0x6,0x7}

Current OBT is = 100024

Current OBT is = 100025
```

Figura 5.5: Se inyecta el telemando TC[3,6]

```
Current OBT is = 100026

Current OBT is = 100027

Current OBT is = 100028
GSS Rx TM [3,25] SID 10 Param Values {0x5,0x6,0x7}

Current OBT is = 100029

Current OBT is = 100030

GSS Tx TC[3,5] FT_UAH_ASW_ICU_SERV_3_0055 step 0, Enable SID 11
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [1,7]

Current OBT is = 100031

Current OBT is = 100032
GSS Rx TM [3,25] SID 10 Param Values {0x5,0x6,0x7}

Current OBT is = 100033

Current OBT is = 100034

Current OBT is = 100035
GSS Rx TM [3,25] SID 11 Param Values {14,15}
GSS Rx TM [3,25] SID 10 Param Values {0x5,0x6,0x7}

Current OBT is = 100036
```

Figura 5.6: Se inyecta el telemando TC[3,5]

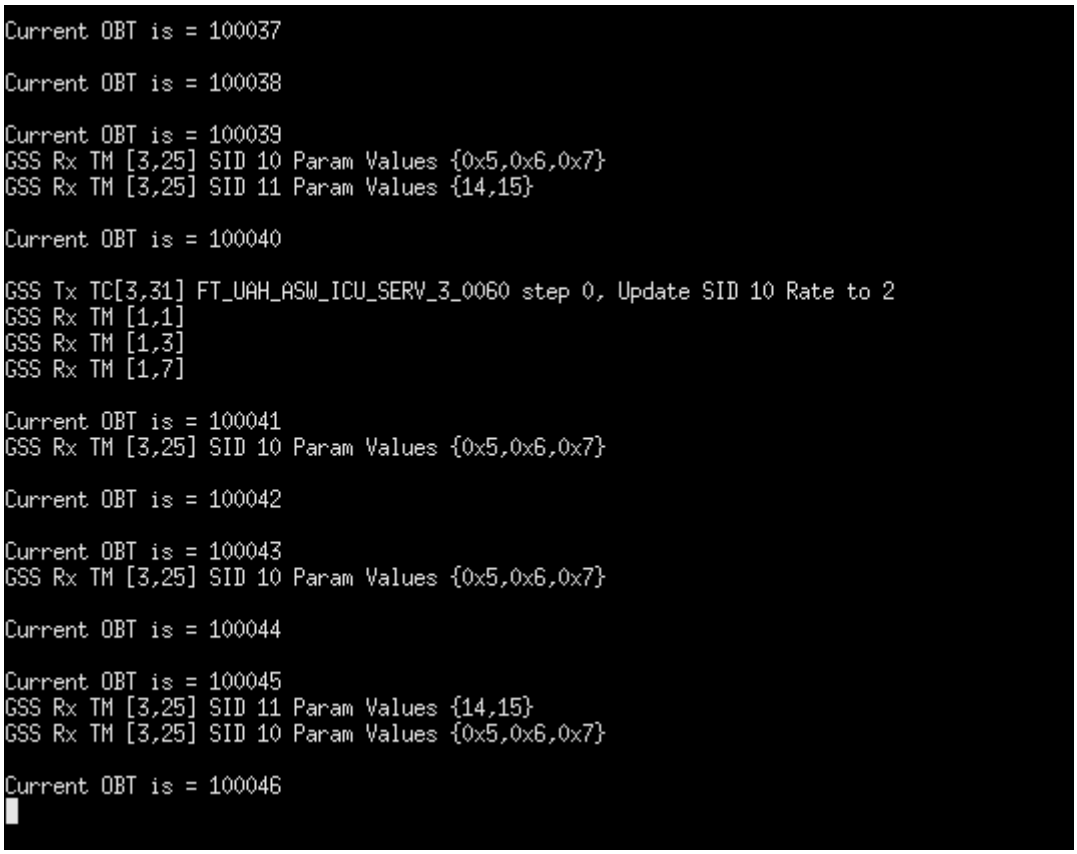


Figura 5.7: Se inyecta el telemando TC[3,31]

5.3 Validación de los servicios PUS 20

Dentro de los servicios PUS, el encargado de administrar parámetros es el número 20, mediante el uso del *System Data Pool*. En concreto, se van a probar dos telemandos: TC[20,3], el cual establece valores a los parámetros indicados, y TC[20,1], que pide el valor de un parámetro en concreto. También se probará la telemetría TM[20,2], cuya función debe ser devolver el valor del parámetro (nace como respuesta a TC[20,1]).

Listado 5.3: Validación de los servicios PUS 20

```
...
#define OBT_START_FT_ST20 (OBT_AFTER_POWER_ON + 0)
#define FT_0070_TIME_step0 (OBT_START_FT_ST20 + 10)
#define FT_0070_TIME_step1 (OBT_START_FT_ST20 + 15)

EmuGSS_TCProgram20_3_uint8 prog_FT_0070_step_0(FT_0070_TIME_step0,
    "FT_UAH_ASW_ICU_ST20_SET_GET_PARAM_0070 step 3, Update PID 15 to 99", 15, 99);

EmuGSS_TCProgram20_1     prog_FT_0070_step_1(FT_0070_TIME_step1,
    "FT_UAH_ASW_ICU_ST20_SET_GET_PARAM_0070 step 4, Get PID 15", 15);
```

La macro que construye el telemando TC[20,3] necesita que se le adjunte el PID al que se le va a modificar el valor, junto con el valor nuevo; mientras que la segunda macro obtiene como parámetro el PID.

5.3.1 Resultado esperado

Tabla 5.3: Telemandos y telemetrías esperados

Caso	Paso	Tiempo	Telemando y datos	Efecto esperado
0070	0	100010	TC[20,3]: N=1, PID=15	El PID 15 pasa a valer 99
0070	1	100015	TC[20,1]: N=1, PID=15	Se recibe una TM[20,2] con parámetro el valor 99

Para las pruebas, se va a tomar el parámetro con PID 15 (ver tabla 5.3). N=1 indica un único parámetro. El PID ocupa 16 bits; N y el valor de este parámetro ocupan 8 bits. El PID 15 es un parámetro válido de tipo uint8_t y permite la escritura mediante telemando. La inicialización del emulador le asigna inicialmente el valor 15.

La primera prueba se lanzará en el segundo 100010. En el código 5.3 se construye dicha macro en la línea 6. Se debe indicar en la construcción del paquete y por parámetro el PID que se quiere modificar (el número 15) y el nuevo valor que se requiere que adopte el parámetro: el 99. El sistema debe responder con la secuencia TM[1,1], TM[1,3] y TM[1,7].

Cinco segundos más tarde, se enviará un TC[20,1] con el objetivo de obtener el valor del parámetro con el PID 15. El sistema debería devolver TM[1,1], TM[1,3], TM[20,2] con el nuevo valor: 99 y TM[1,7]. TM[20,2] debe contener: número de parámetros (N) = 1, identificador (PID) = 15 y valor (uint8_t) = 99 (0x63).

En total, se esperan siete telemetrías asociadas a los dos telemandos: tres confirmaciones para la escritura y cuatro respuestas para la lectura. La comprobación decisiva es que la TM[20,2] devuelva PID 15 = 99.

5.3.2 Resultados de la validación

Explicada la validación que se va a realizar, es el momento de mostrar el resultado (ver figura 5.8). Los segundos del OBT en los que se mandan ambos comandos se corresponden a los que aparecen en el código. La construcción de los telemandos y telemetrías es correcta:

- Cuando se envía TC[20,3] al sistema, este genera e imprime por pantalla TM[1,1], TM[1,3] y TM[1,7], dando a entender que la construcción y ejecución del comando ha sido exitosa. En el momento en que el sistema ejecuta el comando, debe ser capaz de cambiar el valor de dicho parámetro.

- Se puede comprobar que el sistema fue capaz de modificar el valor a 99 del parámetro con PID 15. El sistema ha devuelto TM[20,2] con el nuevo valor del parámetro a la consulta realizada en el comando.

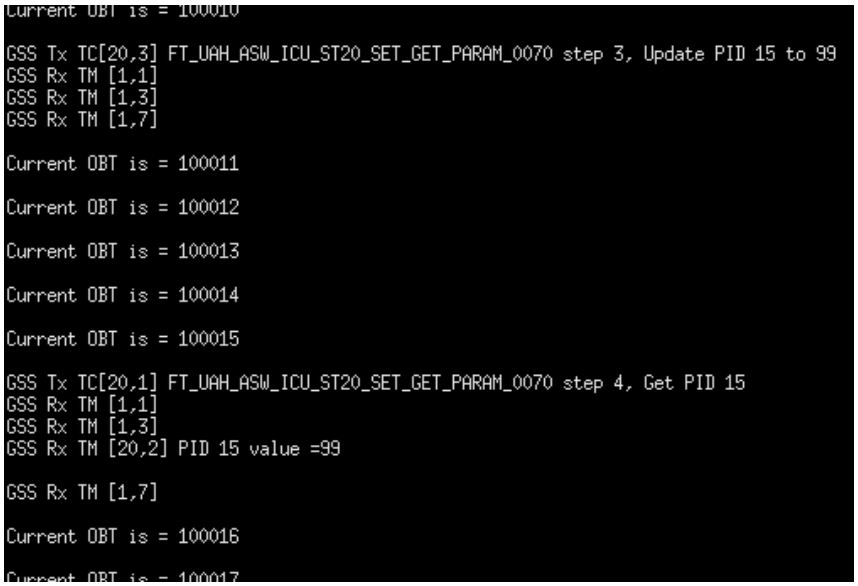


Figura 5.8: Se inyectan los telemandos TC[20,3] y TC[20,1]

Listado 5.4: Validación de FDIR

```
#define OBT_START_FT_ST19 (OBT_AFTER_POWER_ON + 60)
#define FT_0080_TIME_step0 (OBT_START_FT_ST19 + 2)
#define FT_0080_TIME_step1 (OBT_START_FT_ST19 + 4)
#define FT_0080_TIME_step2 (OBT_START_FT_ST19 + 6)
#define FT_0080_TIME_step3 (OBT_START_FT_ST19 + 8)
#define FT_0080_TIME_step4 (OBT_START_FT_ST19 + 10)
#define FT_0080_TIME_step5 (OBT_START_FT_ST19 + 12)
#define FT_0080_TIME_step6 (OBT_START_FT_ST19 + 20)

EmuGSS_TCProgram20_3_uint8 prog_FT_0080_step_0A(FT_0080_TIME_step0,
    "FT_UAH_ASW_ICU_Monitoring_0110 step 0, Set PID 15 to 5",15,5);

EmuGSS_TCProgram12_5_Limit_UINT8 prog_FT_0080_step_1(FT_0080_TIME_step1,
    "FT_UAH_ASW_ICU_Monitoring_0110 step 1, Config PMODID 0 for monitoring PID 15",
    0, 15, 1, 2, 1, 0x4001, 20, 0x4002);

EmuGSS_TCProgram19_1_Action_2_1 prog_FT_0080_step_2(FT_0080_TIME_step2,
          "FT_UAH_ASW_FDIR_0080 step 2, 0x4002 Action is Device  Off",
          0x4002, DeviceCommandOff);

EmuGSS_TCProgram5_5 prog_FT_0080_step_3A(FT_0080_TIME_step3,
    "FT_UAH_ASW_FDIR_0080 step 3A, Enable Ev 0x4001",
    0x4001);

EmuGSS_TCProgram5_5 prog_FT_0080_step_3B(FT_0080_TIME_step3,
    "FT_UAH_ASW_FDIR_0080 step 3B, Enable Ev 0x4002",
    0x4002);

EmuGSS_TCProgram19_4 prog_FT_0080_step_4(FT_0080_TIME_step4,
          "FT_UAH_ASWFDIR_0080 step 4, 0x4002 Action enabled",
          0x4002);

EmuGSS_TCProgram12_1 prog_FT_0080_step_5(FT_0080_TIME_step5,
    "FT_UAH_ASW_FDIR_0080 step 3,   ", 0);

EmuGSS_TCProgram20_3_uint8 prog_FT_0080_step_6(FT_0080_TIME_step6,
    "FT_UAH_ASW_Monitoring_0110 step 5, Update PID 15 to 99",15,99);
```

5.4 Validación de FDIR

En una nave espacial, el tratamiento de fallos juega un papel drástico. Es por eso que es necesario el uso de FDIR. Se explicó en la sección 4.4 la función del componente HKFDIRMng para el manejo de la robustez del proyecto. En esta validación y la siguiente se va a comprobar si es capaz de recuperarse ante fallos. El servicio PUS 19 es el encargado de ejecutar de forma autónoma acciones cuando ocurren eventos específicos a bordo. De esta manera, el componente HKFDIRMng generará los telemandos relacionados a este servicio y se los enviará al componente TCManager a través de una interfaz asíncrona.

En esta validación también entrará en juego el servicio PUS 12, el cual proporciona la capacidad de monitorear parámetros o grupos de parámetros a bordo y reaccionar ante las infracciones de las condiciones de monitorización relacionadas mediante la generación de eventos.

Es importante destacar el papel que tiene el servicio 05 sobre esta prueba. La telemetría de dicho servicio muestra, a través de informes, el nivel de gravedad de la anomalía detectada, siendo TM[5,2] nivel bajo, TM[5,3] nivel medio y TM[5,4] nivel alto. Cuando el evento provoca una anomalía, el sistema lanza alguna de estas telemetrías y ejecuta la acción oportuna para detener el problema.

El objetivo de la última validación es que el sistema FDIR sepa detectar cuando un parámetro supera su límite máximo y ejecute automáticamente un comando de apagado del dispositivo.

5.4.1 Resultado esperado

Hay ocho telemandos, además del TC[2,1], que se ejecuta automáticamente tras la detección de un fallo:

- En el segundo 100183, se recibirá un telemando TC[20,3] para modificar el valor a 5 del parámetro con PID 15.
- Dos segundos más tarde, se definirá el Parameter Monitoring ID (PMONID) 0 para monitoriear el parámetro con PID 15. Se le indican varios parámetros extra, en orden: PMONID = 0, PID = 15, Intervalo = 1, Repetición = 2, Límite inferior=1, Identificador del evento=0x4001, Límite superior=20, Identificador del evento=0x4002.
- La macro **EmuGSS_TCProgram19_1_Action_2_1** utiliza TC[19,1] para asociar un evento (**0x4002**) con una acción de TC[2,1]. Esto significa que el telemando indica que el dispositivo va a apagarse (**DeviceCommandOff**). Aunque el telemando se haya enviado, este no se activa hasta que no se ejecute más tarde TC[19,4]. Es decir, se ha construido la evento-acción, pero tanto el evento como la acción están deshabilitadas.
- Para activar el evento, se hará uso del telemando TC[5,5]. En la macro simplemente hay que indicar el código del evento concreto a activar.
- Para habilitar la acción, entra en escena TC[19,4]. Al igual que TC[5,5], hay que indicar el identificador del evento.
- La macro **EmuGSS_TCProgram12_1** habilita las definiciones de monitoreo de parámetros creados mediante TC[12,5].
- Se vuelve a actualizar el valor del parámetro con PID 15, esta vez a 99, usando TC[20,3].

Aquellos telemandos exitosos deben recibir como respuesta la secuencia de telemetrías TM[1,1], TM[1,3] y TM[1,7]. Esto da un total de 24 paquetes de verificación para los ocho comandos programados, sin contar la telemetría que se activa al inicio y al final de la ejecución del comando TC[2,1].

Tabla 5.4: Telemandos y telemetrías esperados

Caso	Paso	Tiempo	Telemando y datos	Efecto esperado
0080	0	100062	TC[20,3]: PID=15, valor=5	Establecer el parámetro PID 15, de 8 bits sin signo, en 5; inicialmente dentro de los límites
0080	1	100064	TC[12,5]: PMONID=0, PID=15, Intervalo=1, Repetición=2, Límite inferior=1, Identificador del evento=0x4001, Límite superior=20, Identificador del evento=0x4002	Define el monitor de límites PMONID 0 para el PID 15: intervalo de 1 ciclo de monitorización, repetición 2, límite inferior 1, límite superior 20 para ambos eventos. Desactivado inicialmente
0080	2	100066	TC[19,1]: Identificador del evento=0x4002, Comando del dispositivo=DeviceCommandOff,	Asocia el evento 0x4002 con un TC[2,1] integrado, que ordena que el dispositivo se apague en la dirección 0x80000900. La instalación de la acción no implica su ejecución
0080	3A-3B	100068	Dos TC[5,5]: Identificador del evento=0x4001, Identificador del evento=0x4002	activan eventos 0x4001 y 0x4002
0080	4	100070	TC[19,4]: Identificador del evento=0x4002	activa la acción de recuperación asociada al código 0x4002
0080	5	100072	TC[12,1]: PMONID=0	habilita el monitor PMONID 0
0080	6	100080	TC[20,3]: PID=15, valor=99	establece el PID 15 = 99, lo que provoca un fallo de límite superior

Una vez se ejecute TC[12,1], el proyecto imprimirá por pantalla los informes del monitor PMONID 0 definido por TC[12,5]. Los informes deben aparecer dentro de la telemetría TC[12,12]. Se tiene que activar un informe justo después de ejecutar TC[12,1] y después de TC[20,3], momento en el cual el sistema detecta que el valor del parámetro con identificador 15 ha cambiado su valor a 99 (se recuerda que se ha creado una alerta en TC[12,5], la cual avisa cuando el valor de dicho parámetro aumenta más allá de 20). Como el aumento de valor es mayor que el límite máximo, se deberá mostrar un informe de gravedad alta (TM[5,4]), seguido del apagado del dispositivo (TC[2,1]) y la confirmación de inicio y cierre exitoso de la ejecución (TM[1,3] y TM[1,7]).

5.4.2 Resultados de la validación

En la imagen 5.9 se puede concluir que los telemandos TC[20,3], TC[12,5], TC[19,1], TC[5,5], TC[19,4] y TC[12,1] se envían y ejecutan correctamente por la aparición de sus respectivas secuencias de telemetrías TM[1,1], TM[1,3] y TM[1,7].

Tal como se explicó en la subsección del resultado esperado, la ejecución de TC[12,1] genera la telemetría TM[12,12] (ver figura 5.10). Dicha telemetría nos indica que el valor anterior del parámetro con

```
GSS Tx TC[20,3] FT_UAH_ASW_FDIR_0080 step 0, Set PID 15 to 5
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [1,7]

Current OBT is = 100063

Current OBT is = 100064

GSS Tx TC[12,5] FT_UAH_ASW_FDIR_0080 step 1, Configure PMONID 0 for monitoring PID 15
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [1,7]

Current OBT is = 100065

Current OBT is = 100066

GSS Tx TC[19,1] FT_UAH_ASW_FDIR_0080 step 2, 0x4002 Action is Device  Off
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [1,7]

Current OBT is = 100067

Current OBT is = 100068

GSS Tx TC[5,5] FT_UAH_ASW_FDIR_0080 step 3A, Enable Ev 0x4001

GSS Tx TC[5,5] FT_UAH_ASW_FDIR_0080 step 3B, Enable Ev 0x4002
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [1,7]
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [1,7]

Current OBT is = 100069

Current OBT is = 100070

GSS Tx TC[19,4] FT_UAH_ASW_FDIR_0080 step 4, 0x4002 Action enabled
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [1,7]

Current OBT is = 100071

Current OBT is = 100072

GSS Tx TC[12,1] FT_UAH_ASW_FDIR_0080 step 5, Enable PMONID 0
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [1,7]
```

Figura 5.9: Inyección de los telemandos TC[20,3], TC[12,5], TC[19,1], TC[5,5], TC[19,4] y TC[12,1]

```
Current OBT is = 100073
GSS Rx TM [12,12] PMONID = 0 PID 15 Check Limits value = 5 limit = 1 prev status Unchecked current status Within Limits OBT = 100073


Current OBT is = 100074

Current OBT is = 100075

Current OBT is = 100076

Current OBT is = 100077

Current OBT is = 100078

Current OBT is = 100079

Current OBT is = 100080

GSS Tx TC[20,3] FT_UAH_ASW_FDIR_0080 step 6, Update PID 15 to 99
GSS Rx TM [1,1]
GSS Rx TM [1,3]
GSS Rx TM [1,7]

Current OBT is = 100081
GSS Rx TM [12,12] PMONID = 0 PID 15 Check Limits value = 99 limit = 20 prev status Within Limits current status Above High Limit OBT = 100080

GSS Rx TM [5,4] High Severity Anomaly EvID 0x4002  PID 15 = 99 High Limit = 20

GSS Rx TM [1,3]
Device Off
GSS Rx TM [1,7]

Current OBT is = 100082

Current OBT is = 100083
```

Figura 5.10: Salida esperada

PID 15 es 5, superior al límite inferior (1). También nos indica el identificador PMONID y el segundo en el cual se realizó el monitoreo.

Una vez se ejecuta TC[20,3] y cambia el valor contenido en el parámetro PID, vuelve a aparecer TM[12,12], pero esta vez con el valor actualizado a 99 (muy superior al límite máximo de 20). Se puede comprobar que el sistema etiqueta tal subida como muy alta, motivo por el que se activa TM[5,4], el cual avisa de que se ha detectado una anomalía de gravedad superior en el cambio de valor. Es entonces cuando se ejecuta TC[2,1], con el objetivo de paliar la incidencia. Se puede comprobar que se imprime por pantalla justo antes y después de la ejecución de TC[2,1] la notificación de inicio y cierre de la ejecución del comando.

5.5 Conclusiones

Después de ejecutar cada una de las pruebas sobre el sistema y analizar su salida por pantalla, se puede concluir que la construcción del nuevo sistema en el programa TASTE ha sido todo un éxito. Como se ha podido comprobar, las combinaciones de telemetrías mostradas por pantalla son correctas según el estándar estipulado por los servicios PUS. También se ha demostrado que el proyecto desarrollado en este Trabajo Fin de Máster reacciona y evoluciona ante los comandos que se le envían (objetivo **O5**).

Capítulo 6

Conclusiones y líneas futuras

Este Trabajo Fin de Máster ha adaptado la estructura empleada en el modelo EDROOM a un nuevo sistema planteado en TASTE. Se ha intentado mantener la arquitectura lo más parecida a la original, pero, por cuestiones de diferencia de diseño del procesador de la máquina virtual con respecto al buscado simular, se produjeron ciertos cambios que se explicarán a continuación.

La mayor variación que sufrió el sistema fue la creación del componente Legacy. Su aparición se debe al hecho de que el resto de los componentes no eran capaces de realizar llamadas de procedimiento a una función perteneciente a ellos mismos. Como se ha ido explicando, el proceso interno de los componentes TCManager, HKFDIR y BKGExec ejecutan funciones internas para realizar operaciones propias de su funcionalidad. A través de SDL se debían realizar las llamadas a los procedimientos antes de cambiar de estado interno. Se probó a incluir cada uno de los componentes una implementación de C++, además de la que contaban de SDL. Sin embargo, se comprobó que el componente no era capaz de alinear la llamada con la función. También se probó a convertir los componentes en *nested functions*, pero aumentaba la complejidad del proyecto. Finalmente, se llegó a la solución que finalmente se implementó: la creación de este nuevo componente al que se deriven las funciones internas del resto. Así, cada vez que un componente necesitase ejecutar una de sus funciones, tan solo debe realizar la llamada al procedimiento de dicha función. Dicha llamada se hace de manera protegida o *protected* (mediante mensaje síncrono).

Otro cambio con respecto al proyecto lanzado sobre el modelo EDROOM es la forma en la que se ejecuta. Inicialmente, el software de partida estaba preparado para su ejecución en una plataforma LEON3 con el sistema operativo RTMES. La versión actual integra la lógica de los servicios PUS en un despliegue TASTE que se ejecuta de forma nativa sobre Linux x86-64. Para ello, dispone de una capa de adaptación que proporcionan las primitivas de temporización y sincronización necesarias en este entorno. Por tanto, esta ejecución constituye una adaptación funcional del software original, por lo que los resultados mostrados no permiten validar directamente el comportamiento del sistema sobre LEON3/RTEMS ni su interacción con el hardware de destino.

Las funciones que forman parte del código de Legacy han tenido que ser adaptadas debido a que la lógica original no se cumplía en el nuevo proyecto. Al haber cambiado el entorno de ejecución a Linux x86-64, había que modificar las funciones para que pudiesen ser compiladas y tratadas correctamente.

La decisión tomada con respecto al tipo de interfaces a utilizar entre los componente se basa en la necesidad por obtener respuestas de manera directa o no. Los canales de comunicación entre Legacy y el resto de componentes son síncronos debido a que se necesitaba de la rapidez de ambos elementos para hacer el tratamiento. Si TCManager pide a Legacy que gestione un telemando, TCManager necesita que se produzca el intercambio de datos al instante.

Los canales asíncronos entre los componentes TCManager, HKFDIRMng y BKGTCExec demuestran que no es necesario que la interlocución tenga que ser instantánea. Cuando TCManager envía un telemando a BKGTCExec, por ejemplo, está transfiriendo la administración de dicho paquete. En ese momento, TCManager ya no quiere saber nada más de su gestión.

Durante los capítulos que abarcan esta documentación, se han ido cumpliendo todos los objetivos planteados en este trabajo. En el capítulo anterior, se ha comprobado el correcto funcionamiento del sistema a la hora de validar que se cumplen las prioridades asignadas a cada uno de los actuadores (objetivo **O4**) y en la construcción, manejo e intercambio de telemando y telemetrías (objetivo **O5**).

También se ha demostrado a lo largo de esta memoria que, los objetivos de desarrollo del nuevo modelo y documentación (**O1**, **O2**, **O3** y **O6**), se han consumado.

Entre las eventuales vías de estudio que pueden sobrevenir, se podría rediseñar la arquitectura del sistema de muchas maneras. Como hemos comentado, se decidió crear el componente Legacy, no tanto como una solución real, sino como una temporal. El hecho de que exista un componente nuevo cuyo único objetivo es ejecutar funciones internas de otros componentes, deja en evidencia una posible fuga en el planteamiento.

Otra vía de desarrollo sería volver a implementar el proyecto como una plataforma LEON3 con el sistema operativo RTEMS. Por ende, la simulación realizada conseguiría ser más fiel a las tecnologías que realmente se utilizan.

Por último, la implementación de un análisis de planificabilidad mediante Modeling and Analysis Suite for Real-Time Applications (MAST) nos permitiría conocer si el sistema actual es eficiente en cuestiones de tiempo. En las sesiones de laboratorio de la asignatura, se llegó a realizar dicho análisis sobre el modelo EDROOM. Uno de los puntos fuertes de TASTE es el hecho de que exista la opción de poder ejecutar la herramienta MAST dentro del propio programa. Conseguir aprobar el análisis demostraría que la construcción del sistema no solo cumple los objetivos de intercambio de telemandos y telemetrías de servicios PUS, sino no que además permitiría conocer si el sistema es eficiente en términos temporales.

Bibliografía

[1] I. Dragomir, C. Redondo, T. Jorge et al., "Model-checking of space systems designed with TASTE/SDL", en *MODELS '22: ACM/IEEE 25th International Conference on Model Driven Engineering Languages and Systems*, Montreal Quebec, Canada: ACM, oct. de 2022, págs. 237-246. DOI: 10.1145/3550356.3561541. dirección: https://hal.science/hal-04106251.

[2] A. Modenini y B. Ripani, "A Tutorial on the Tracking, Telemetry, and Command (TT&C) for Space Missions", *IEEE Communications Surveys & Tutorials*, vol. 25, n.º 3, págs. 1510-1542, 2023. DOI: 10.1109/comst.2023.3287431. dirección: https://doi.org/10.1109/comst.2023.3287431.

[3] E. C. for Space Standardization, *ECSS-E-ST-50C Rev.2 – Communications (5 December 2024)*, en-US, dic. de 2024. dirección: https://ecss.nl/standard/ecss-e-st-50c-rev-1-communications-5-december-2024/.

[4] E. C. for Space Standardization, *ECSS-E-AS-50-24C Rev.1– Adoption Notice of CCSDS 231.0-B-4, TC Synchronization and Channel Coding (13 January 2023)*, en-US, ene. de 2023. dirección: https://ecss.nl/standard/ecss-e-as-50-24c-rev-1-adoption-notice-of-ccsds-231-0-b-4-tc-synchronization-and-channel-coding/.

[5] E. C. for Space Standardization, *ECSS-E-AS-50-21C Rev.2– Adoption Notice of CCSDS 131.0-B-5, TM Synchronization and Channel Coding (5 December 2024)*, en-US, dic. de 2024. dirección: https://ecss.nl/standard/ecss-e-as-50-21c-rev-1-adoption-notice-of-ccsds-131-0-b-5-tm-synchronization-and-channel-coding-5-december-2024/.

[6] Y. Zhan, G. Zeng y X. Pan, "Networked TT&C for mega satellite constellations: A security perspective", *China Communications*, vol. 19, n.º 9, págs. 58-76, sep. de 2022. DOI: 10.23919/jcc.2022.09.006. dirección: https://doi.org/10.23919/jcc.2022.09.006.

[7] T. consultative Committee for Space Data Systems, *CCSDS 133.0-B-2 - Space Packet Protocol (June 2020)*, en, jun. de 2020. dirección: https://ccsds.org/publications/allpubs/entry/3264/.

[8] E. C. for Space Standardization, *ECSS-E-ST-70-41C – Telemetry and telecommand packet utilization (15 April 2016)*, en-US, abr. de 2016. dirección: https://ecss.nl/standard/ecss-e-st-70-41c-space-engineering-telemetry-and-telecommand-packet-utilization-15-april-2016/.

[9] J.-F. Kaufeler, *The ESA standard for telemetry and telecommand packet utilisation: PUS*, en, nov. de 1994. dirección: https://ntrs.nasa.gov/citations/19950011180.

[10] E. C. for Space Standardization, *ECSS-E-ST-70-01C – Spacecraft on-board control procedures (16 April 2010)*, en-US, abr. de 2010. dirección: https://ecss.nl/standard/ecss-e-st-70-01c-on-board-control-procedures/.

[11] M. Perrotin, *Overview · Wiki · TASTE / taste-setup · GitLab*, en, mar. de 2025. dirección: https://gitlab.esa.int/taste/taste-setup/-/wikis/Overview.

[12] U. I. de Telecomunicaciones, *SDL Forum Society | Home page*. dirección: https://www.sdl-forum.org/.

[13] M. Perrotin, $Technical_{t}opic_{O}penGEODE_{a}n_{S}DL_{e}ditor_{f}or_{T}ASTE$ůWikiů$TASTE/taste-setup$ů$GitLab$, en, mar. de 2025. dirección: https://gitlab.esa.int/taste/taste-setup/-/wikis/Technical_topic_OpenGEODE_an_SDL_editor_for_TASTE.

[14] M. Perrotin, $Detailed_{S}DL_{t}utorial$ů$Wiki$ů$TASTE/taste-setup$ů$GitLab$, en, jul. de 2026. dirección: https://gitlab.esa.int/taste/taste-setup/-/wikis/Detailed_SDL_tutorial.

[15] M. Perrotin, E. Conquet, J. Delange y T. Tsiodras, *TASTE: An open-source tool-chain for embedded system and software development*, en, feb. de 2012. dirección: https://hal.science/hal-02191871.

[16] G. Nuefeld e Y. Yang, *The design and implementation of an ASN.1-C compiler*$_{1}$990, en-US, oct. de 1990. dirección: https://ieeexplore.ieee.org/abstract/document/60300.

Universidad de Alcalá
Escuela Politécnica Superior

ESCUELA POLITECNICA
SUPERIOR

